



Princess Sumaya University for Technology
King Hussein School for Computing Sciences

Smart Student Schedule System

Prepared By:

Dana Rami Tawfiq Salem	20220177
Laila Omar Talal Sadideen	20220432
Yazan Allam Ahmad Fani	20220919
Yousef Sameer Ahmad Saddouq	20220187

Supervised By:

Dr. Samer Sawalha

Project Submitted in partial fulfillment for the degree of Bachelor of Science in
Computer Science

Second Semester - 2026

Declaration of Originality

We, the undersigned, hereby declare that this document has been entirely written and prepared by the project team members listed below.

All materials, ideas, or text taken from external sources have been clearly acknowledged and properly cited in accordance with academic and ethical standards. Every quotation, illustration, image, table, or figure that is not our original work is accompanied by a complete and accurate reference indicating its source.

We further affirm that this project is the result of our own effort, research, and development, and has not been submitted, in whole or in part, for any other course, degree, or institution.

We fully understand that any instance of plagiarism, data fabrication, or misrepresentation constitutes a violation of the academic integrity policy of Princess Sumaya University for Technology (PSUT) and may result in disciplinary action as per the university's regulations.

Names and signatures of Team Members:

Student Name	University ID	Signature	Date
Dana Rami Tawfiq Salem	20220177		/ / 2026
Laila Omar Talal Sadideen	20220432		/ / 2026
Yazan Allam Ahmad Fani	20220919		/ / 2026
Yousef Sameer Ahmad Saddouq	20220187		/ / 2026

Acknowledgments

First, we sincerely thank Princess Sumaya University for Technology (PSUT) and the King Hussein School of Computing Sciences for their valuable guidance and support.

We would like to thank our project supervisor, Dr. Samer Sawalha, for his guidance and the time he dedicated to reviewing our work. His comments and directions helped us stay focused and improve the quality of our project.

Our thanks extend to Dr. Ammar Odeh and all faculty members who were always willing to help whenever we had questions or needed clarification during the semester.

Finally, we want to thank our families and friends for their patience and encouragement. Their support made this project possible.

Summary

Each semester, students start stressing about the course registration period. Manually constructing the schedule from various sections for each course while focusing on lessening the days of attendance and keeping an eye on the difficulty of the timetable is very complex and stressful. The existing PSUT registration system is very rigid and requires students to make their own timetable and choose sections that are suitable for their routine. Students tend to make plans days before the registration period and start writing possible schedules and choose the most suitable one. This process takes plenty of time, and most of the time, students don't get what they want. Here, our system comes and assists students with this problem.

The Student Smart Scheduler aims to solve this problem by optimizing the creation of the university semester schedules. The goal is to generate an algorithm that creates clash-free and personalized schedules. The system is designed to take a student's required courses with the total hours of the semester and a set of preferences as input from the user, producing all possible schedules that satisfy the user's constraints, ranked from most fit to least fit.

First, the user starts inputting the list of required courses for the semester. Then, the user chooses the set of preferences, including:

- Time Range: Specifying earliest and latest class times.
- Break times: Controlling the desired break times by specifying the min and max time between classes.
- Attendance days: Selecting preferred days to attend on campus and choosing when to have off-days.
- Instructor Preference: Giving priority to sections with specific instructors.

The system uses API integration for courses' data from the university database. It was developed using React, .NET Core. The system builds an algorithm that creates all possible, clash-free, schedules according to the filters the student inputs.

The Student Smart Scheduler provides a necessary solution to a very common challenge in the student's university life. It efficiently acts as a medium between the student's preferences and the university's course offerings. So it not only provides a clash-free schedule, but it also enhances the student's control over his/her academic experience, which definitely improves the student's satisfaction and his/her performance each semester.

List of Abbreviations

API: Application Programming Interface

DB: Database

DBMS: Database Management System

HTML: HyperText Markup Language

HTTP: Hypertext Transfer Protocol

HTTPS: Hypertext Transfer Protocol Secure

IIS: Internet Information Services

PDF: Portable Document Format

QR: Quick Response (Code)

REST: Representational State Transfer

SIS: Student Information System

SQL: Structured Query Language

TLS: Transport Layer Security

UI: User Interface

UX: User Experience

Table of Contents

Chapter 1 Introduction.....	12
1.1 Overview	12
1.2 Problem Statement	12
1.3 Related Work	13
1.4 Document Outline	18
Chapter 2 Project Plan	19
2.1 Project Deliverables	19
2.2 Project Tasks	19
2.3 Roles and Responsibilities	24
2.4 Risk Assessment	25
2.5 Cost Estimation.....	26
2.6 Project Management Tools	27
Chapter 3 Requirements Specification	28
3.1 Stakeholders	28
3.2 Platform Requirements	29
3.3 Functional Requirements.....	30
3.4 Non-Functional Requirements.....	32
3.5 Other Requirements	33
Chapter 4 System Design	34
4.1 Logical Model Design	34
4.1.1 <i>Architecture Overview</i>	34
4.1.2 <i>Required UML Diagrams</i>	35
1. Architecture Diagram	35
2. Class Diagram.....	35
3. Object Diagram.....	37
4. Use Case Diagram	40
5. Activity Diagram	46
6. Sequence Diagram.....	50
7. State Transition Diagram.....	54
4.1.3 <i>Data Design – ERD & Database Schema</i>	56
4.1.4 <i>User Interface Design & Navigation</i>	61
4.2 Physical Model Design	82
4.2.1 <i>Reports Design</i>	82
4.2.2 <i>User Interface Design (Physical Specs)</i>	82
4.2.3 <i>Database Design (3NF)</i>	84

Table of Figures

FIGURE 2.1 GANTT CHART SHOWING THE TASKS COMPLETION AND THEIR DEPENDENCIES FOR ANALYSIS AND DESIGN PHASE.....	22
FIGURE 2.2 GANTT CHART SHOWING THE TASKS COMPLETION AND THEIR DEPENDENCIES FOR THE IMPLEMENTATION PHASE	23
FIGURE 4.1 ARCHITECTURE DIAGRAM SHOWING THE COMPONENTS OF THE SYSTEM AND HOW THEY'RE RELATED.....	35
FIGURE 4.2 CLASS DIAGRAM SHOWING THE MAIN COMPONENTS OF THE SYSTEM AS CLASSES WITH ATTRIBUTES AND METHODS.	36
FIGURE 4.3 OBJECT DIAGRAM SHOWING INSTANCES FOR THE STUDENT, INSTRUCTOR, COURSE, SECTION, AND ADDED COURSE CLASSES.....	37
FIGURE 4.4 OBJECT DIAGRAM SHOWING INSTANCES OF THE STUDENT, FILTER, GENERATED SCHEDULE, AND SECTION CLASSES.....	38
FIGURE 4.5 OBJECT DIAGRAM SHOWING AN INSTANCE OF THE GENERATED SCHEDULE CLASS AND FAVORITE CLASS.....	39
FIGURE 4.6 USE CASE DIAGRAM SHOWING THE STUDENT'S MAIN INTERACTIONS WITH THE SMART SCHEDULING SYSTEM.....	40
FIGURE 4.7 USE CASE DIAGRAM ILLUSTRATING HOW THE STUDENT CONFIGURES SCHEDULE PREFERENCES.....	43
FIGURE 4.8 USE CASE DIAGRAM SHOWING HOW THE STUDENT REVIEWS AND MANAGES GENERATED SCHEDULES.	44
FIGURE 4.9 ACTIVITY DIAGRAM FOR USER LOGIN AND AUTHENTICATION PROCESS.....	46
FIGURE 4.10 ACTIVITY DIAGRAM FOR OPENING THE SMART SCHEDULE AND LOADING INITIAL DATA	47
FIGURE 4.11 ACTIVITY DIAGRAM FOR SELECTING COURSES AND GENERATING SCHEDULES	48
FIGURE 4.12 ACTIVITY DIAGRAM FOR REVIEWING, FAVORING, AND EXPORTING SCHEDULES	49
FIGURE 4.13 SEQUENCE DIAGRAM SHOWING THE ADD COURSE SCENARIO.....	50
FIGURE 4.14 SEQUENCE DIAGRAM SHOWING THE REMOVE COURSE SCENARIO..	51
FIGURE 4.15 SEQUENCE DIAGRAM SHOWING HOW THE STUDENT CAN GENERATE SCHEDULES.	52
FIGURE 4.16 SEQUENCE DIAGRAM SHOWING THE LOGIN PROCESS.....	53
FIGURE 4.17 STATE DIAGRAM SHOWING HOW THE STUDENT SELECTS AND REMOVES COURSES.	54
FIGURE 4.18 STATE DIAGRAM SHOWING THE DIFFERENT FILTER STATES WITHIN THE SYSTEM.....	54

FIGURE 4.19 STATE DIAGRAM SHOWING THE SCHEDULE GENERATION PROCESS IN THE SYSTEM.	55
FIGURE 4.20 ER DIAGRAM SHOWING THE ENTITIES OF THE SYSTEM, THEIR ATTRIBUTES, AND THEIR RELATIONSHIPS	60
FIGURE 4.21 PAGE TREE DIAGRAM SHOWING THE STUDENT’S FLOW THROUGH THE PAGE.	63
FIGURE 4.22 LOGIN PAGE	64
FIGURE 4.23 REMAINING COURSES TABLE.	65
FIGURE 4.24 SELECTION OF INSTRUCTORS IN THE REMAINING COURSES TABLE.	66
FIGURE 4.25 AFTER SELECTING THE INSTRUCTORS.	67
FIGURE 4.26 ADDED COURSES TABLE THAT SHOWS THE COURSES YOU ADDED.	68
FIGURE 4.27 DROPDOWN MENU DISPLAYING THE INSTRUCTORS YOU SELECTED.	69
FIGURE 4.28 FILTERS TO INPUT THE PREFERRED CONSTRAINTS.	70
FIGURE 4.29 USABLE TIME BOX FOR EASY CHOICE IN THE TIME RANGE FIELD. ..	71
FIGURE 4.30 NUMERIC BOX FOR THE BREAK HOURS FIELD.	72
FIGURE 4.31 NUMERIC BOX FOR THE CREDIT HOURS FIELD.	73
FIGURE 4.32 SCHEDULES GENERATED AFTER CLICKING GENERATE SCHEDULES.	74
FIGURE 4.33 OPTION 2 OF THE GENERATED SCHEDULES.	75
FIGURE 4.34 OPTION 3 OF THE GENERATED SCHEDULES.	76
FIGURE 4.35 SELECTING OPTION 1 AS FAVORITE.....	77
FIGURE 4.36 SELECTING OPTION 2 AS FAVORITE.....	78
FIGURE 4.37 DISPLAYED FAVORITE SCHEDULE 1 WITH EXPORT TO PDF FEATURE (FOR ALL FAVORITES) AND REMOVE FAVORITE FEATURE.....	79
FIGURE 4.38 DISPLAYED FAVORITE SCHEDULE 2 WITH REMOVE FAVORITE FEATURE.	80
FIGURE 4.39 NO SCHEDULE FOUND MESSAGE WHEN NO SCHEDULE IS GENERATED	81
FIGURE 4.40 SCHEMA SHOWING THE ACTUAL PHYSICAL STRUCTURE OF THE SYSTEM’S DATABASE IN THE THIRD NORMAL FORM.....	84

Table of Tables

TABLE 1.1 OVERVIEW OF THE PROBLEM, EXPECTED OUTCOMES, AND OUR TARGET AUDIENCE	13
TABLE 1.2 ANALYZING OTHER APPLICATIONS THAT PROVIDED A SOLUTION TO THE SAME PROBLEM	14
TABLE 1.3 PRESENTS A COMPARISON BETWEEN OUR SMART SCHEDULE SYSTEM AND OTHER EXISTING SCHEDULING APPLICATIONS TO HIGHLIGHT THE SIMILARITIES AND DIFFERENCES IN THEIR MAIN FEATURES.	16
TABLE 1.4 PRESENTS THE MAIN CHAPTERS INCLUDED IN THIS DOCUMENT AND GIVES A BRIEF IDEA OF WHAT EACH CHAPTER CONTAINS.	18
TABLE 2.1 PROJECT DELIVERABLES WITH THEIR DESCRIPTIONS AND FORMATS.	19
TABLE 2.2 PROJECT TASKS, DURATION OF EACH TASK, AND THEIR DEPENDENCIES	20
TABLE 2.3 SHOWS EACH TEAM MEMBER’S ROLES AND RESPONSIBILITIES.	24
TABLE 2.4 RISK ASSESSMENT AND MITIGATION PLAN FOR THE SMART SCHEDULER SYSTEM.....	25
TABLE 2.5 COST ESTIMATION FOR THE PROPOSED SYSTEM	26
TABLE 2.6 SHOWS THE TOOLS WE USED AND THEIR DESCRIPTION.....	27
TABLE 3.1 IDENTIFIED STAKEHOLDERS AND THEIR ROLES	28
TABLE 3.2 REQUIREMENTS THAT MUST BE MET IN THE STUDENT’S PERSPECTIVE, ALONG WITH ITS DESCRIPTION AND STATUS.....	29
TABLE 3.3 REQUIREMENTS THAT MUST BE MET IN THE SERVER’S PERSPECTIVE, ALONG WITH ITS DESCRIPTION AND STATUS.....	29
TABLE 3.4 DETAILED FUNCTIONAL REQUIREMENTS INCLUDING INPUTS, PROCESSES, OUTPUTS, AND PRIORITIES	30
TABLE 3.5 NON-FUNCTIONAL REQUIREMENTS OF THE SYSTEM WITH THEIR DESCRIPTIONS AND EXAMPLES	32
TABLE 3.6 ADDITIONAL SYSTEM REQUIREMENTS BEYOND FUNCTIONAL AND NON-FUNCTIONAL SPECIFICATIONS	33
TABLE 4.1 HIGH-LEVEL ARCHITECTURE OVERVIEW OF SYSTEM LAYERS AND TECHNOLOGIES	34
TABLE 4.2 TYPICAL COURSE OF ACTIONS FOR THE GENERATE SCHEDULES USE CASE	41
TABLE 4.3 TYPICAL COURSE OF ACTIONS FOR THE VIEW GENERATED SCHEDULES USE CASE	45
TABLE 4.4 DATABASE TABLES, AND INFORMATION ABOUT THEIR KEYS.....	56
TABLE 4.5 OVERVIEW OF THE USER INTERFACE SCREENS AND NAVIGATION FLOW	61

TABLE 4.6 REPORT DESIGN SPECIFICATION FOR EXPORTING FAVORITE SCHEDULES AS PDF.....	82
TABLE 4.7 TABLE SHOWING THE PHYSICAL SPECIFICATIONS (UI) OF THE SYSTEM.	82
TABLE 4.8 PHYSICAL SCHEMA NORMALIZATION DETAILS.	85

Chapter 1 Introduction

Overview

Nowadays, Automation and Artificial Intelligence are no longer a luxury, but they have become a necessity. Nearly all students struggle and find it time-consuming to design their university's course schedule. Therefore, we developed a Web application with an easy-to-use interface that eliminates the need to register for courses manually. The Smart Student Scheduling system simplifies this process by letting the students choose their preferred courses with many filters based on their preferences, like preferred attendance days, preferred break times, and even their preferred instructors. Our developed optimization algorithms filter the needs of the students and suggest the matching and non-conflicting schedules based on the previous data and statistics we have from our university's database. After that, students can easily compare the resulting schedules and choose the one that best fits their needs.

Our project comes from the real difficulties students face when planning and registering for their courses. By using our own optimization algorithms, we aim to build a practical software solution that helps students organize their schedules in a more convenient and user-friendly way.

Most of the systems now follow the manual process of course registration, which is not only time-consuming, but it often leads to conflicting courses. Our project's motivation is to eliminate this traditional process and provide an easy-to-use application that helps students find the best schedule that fits their needs.

The significance of our project is that it will benefit both students and the university. For students, it will save their time, reduce the chance of error (conflicting courses), and make them feel special as we personalize the schedules according to their personal preferences. For the university, it will improve the registration process and reduce administrative bottlenecks, so that fewer students will ask the administration staff for help during the registration process.

Problem Statement

Table 1.1 shows the common challenges that PSUT students face when registering for the semesters and creating their timetables. By providing filters for user's preferences and generating clash-free schedules, we save the student's time and effort in his registration process and help him meet his academic needs.

Table 1.1 Overview of the problem, expected outcomes, and our target audience

Aspect	Description
Problem Definition	Many university students find it difficult when they create their schedules every semester. They need to check the remaining courses, compare class times, and make sure that there are no conflicts between lectures. Most of the students create multiple samples to compare options, which leads to confusion and delay. This traditional way does not provide flexibility nor simplicity when making schedule decisions.
Expected Outcomes	A smart web-based scheduling system that enables students to generate different schedules according to their needs and preferences. The system is based on using filters that allow students to choose their preferred days, class times, and other specifications. It will reduce the time and the effort required for organizing schedules manually.
Target Audience	The system targets PSUT students who face challenges when creating their schedules. It will offer a simple and user-friendly interface that makes the process easier. All in all, the system will make the scheduling process faster, organized, and much more convenient for students.

Related Work

Table 1.2 shows how we studied a number of scheduling applications to gain a better understanding of current solutions that are comparable to our system. We will determine the gaps

our project will address and how it will enhance the scheduling experience overall by contrasting them with our system.

Table 1.2 Analyzing other applications that provided a solution to the same problem

System	Description	Strengths	Limitations/Gaps
Coursicle	Coursicle is a website and mobile app that helps university students plan their schedules. It's mainly used in the United States and supports many universities. It allows students to search for courses, check class times, and see if seats are available. Moreover, students can add the courses they want to take and arrange them on a visual timetable.[1]	Simple and easy to use. Provides a visual timetable for organizing classes	Does NOT generate schedules automatically. Does NOT take student preferences into account (preferred days, times, breaks, etc.)
student's schedule - application at Neelain University	Student reg. system where it helps students by installing all courses for a semester, then allows them to add/delete a course/courses according to what they want, and the system checks if it's valid to delete or add a specific course. So, the system is not fully automatic, the student still needs to add and delete on his own.[2]	Simple user interface, reliable and can be a university-standalone reg. system	Very limited features, students must check the required sections and his preferred days of attending manually. Requires more work from the user.
System	Description	Strengths	Limitations/Gaps

YouScheduler	A website used by university students to build their schedules. Supports universities in the United States, including the University of New Hampshire. Allows students to choose courses, add breaks, and view possible schedule combinations.[3]	Simple and easy to use. Helps students organize courses and breaks. Provides different schedule options to compare.	Not connected to the university system (no SIS). Does not offer advanced or personalized filters.
UniScheduler Tool	A scheduling tool that is used by six universities in Australia and New Zealand. It mainly focuses on providing flexible schedules that lead to convenience. It allows students to choose their university (from the six universities provided), and they start adding the courses they want. The preferences provided are: Earliest start time, Latest finish time, Maximum days per week, a preferred online classes button (on/off), Day preferences, Time preference for each day selected, Minimum break duration.[4]	Uses genetic algorithms as a backbone of the system.	Does not use AI tools to rank the generated schedules.

Table 1.3 shows that the PSUT Smart Scheduling System provides a more complete and student-focused solution compared to other existing tools. While most systems offer basic

scheduling functions, our system combines real-time university data with flexible preference-based scheduling, allowing students to make better and more comfortable decisions. This makes the PSUT system more practical, reliable, and better suited to real student needs.

Table 1.3 Presents a comparison between our Smart Schedule System and other existing scheduling applications to highlight the similarities and differences in their main features.

Features	PSUT Schedule System	Coursicle	Smart Scheduler, Neelain University	YouScheduler	UniScheduler Tool
Conflict-Free Schedule Generation	√	√	√	√	√
Day Preference	√			√	√
Time Range	√			√	√
Credit Hours	√				
Break Hours	√			√	√
Instructor Preference	√				
Favorite Schedules	√			√	
Export Schedule (PDF)	√			√	
Real-Time Data Sync	√	√	√		

Features	PSUT Schedule System	Coursicle	Smart Scheduler, Neelain University	YouScheduler	UniScheduler Tool
SIS Integration (University System)	√		√		√
Automatic Schedule Generation	√			√	√
Schedules Ranking	√				

Document Outline

Table 1.4 shows a quick overview of this document's titles and descriptions.

Table 1.4 presents the main chapters included in this document and gives a brief idea of what each chapter contains.

Chapter Title	Description
Chapter 1: Introduction	Presents the project idea, explains the issue of manually creating schedules, and gives background information and a brief review of similar systems.
Chapter 2: Project Management and Planning	Describes the main project goals, the weekly plan, team roles, timeline, and the tools used to complete the work.
Chapter 3: System Requirements	Lists the main system requirements in detail, including both functional and non-functional parts, and defines all users and their roles in the system.
Chapter 4: System Design and Implementation	Explains how the system will work through diagrams and models, showing the structure, database, and the main interfaces of the Smart Schedule System.

Chapter 2 Project Plan

Project Deliverables

Error! Reference source not found. shows the project deliverables. These project deliverables guarantee a successful development and evaluation for the smart scheduling system. By analyzing the source code, documentation, files, DBs, and test reports, these deliverables provide a complete documented solution for our project.

Table 2.1 Project Deliverables with their descriptions and formats.

Deliverable	Description	Format / Notes
Source Code	The complete code for the system, including both the frontend and backend components.	Stored in Git repository.
Documentation	All project documentation, including the project report, code documentation, and system documentation.	PDF and Word formats.
Executable Files	A fully functional web-based scheduling system integrated with the university's portal.	Executable or containerized format.
Database Schema	A designed and implemented database containing courses, students, and scheduling data.	SQL file and ER Diagram.
Test Reports	Detailed testing results showing the system's functionality, performance, and security.	Excel and PDF formats.

Project Tasks

Table 2.2 shows the project tasks, a summarized description, estimated duration of each task, and their dependencies.

Table 2.2 Project tasks, duration of each task, and their dependencies

Task No.	Task Name & Description	Duration (Days)	Dependencies	Phase
T1	Startup Meeting: meeting with our supervisor to discuss the smart scheduling system idea and its main objectives.	1	-	Analysis
T2	Problem Statement and Requirement Analysis: discussing the major challenges that students usually face when creating a schedule manually and gathering system requirements.	4	T1	Analysis
T3	Similar Applications Comparison: comparing existing scheduling systems and identifying how our system will improve it.	2	T2	Analysis
T4	Functional Specification: defining our system's functionalities, such as filtering courses, handling preferences, and generating non-conflicting schedules.	5	T3	Analysis

Task No.	Task Name & Description	Duration (Days)	Dependencies	Phase
T5	Feasibility Study: evaluating & determining the feasibility of our algorithms and AI tools, along with the cost and resources available at PSUT.	4	T4	Analysis
T6	Tools and Technology Selection: choosing the programming languages, frameworks, and AI tools that will be used for our system implementation.	2	T5	Analysis
T7	System Architecture Design: designing diagrams to visually represent the system.	9	T6	Design
T8	Database Design: creating the database schemas.	10	T7	Design
T9	User Interface Design: creating Mock Designs using Figma tool.	5	T8	Design
T10	Algorithm Design: designing the algorithm that generates clash-free timetables and filtering out according to the student's preferences.	11	T8	Implementation
T11	Code Implementation: front-end, Back-end, and AI Integration.	35	T6,T7,T8,T9,T10	Implementation
T12	System Integration: connecting our system to the PSUT portal.	15	T7,T8,T9,T10,T11	Implementation

Gantt Chart

Figure 2.1 and Figure 2.2 showing the tasks completion throughout the project. Divided into three phases: Analysis, Design, and Implementation. Each phase with the estimated start date and duration. We can also see clearly the dependencies between tasks.

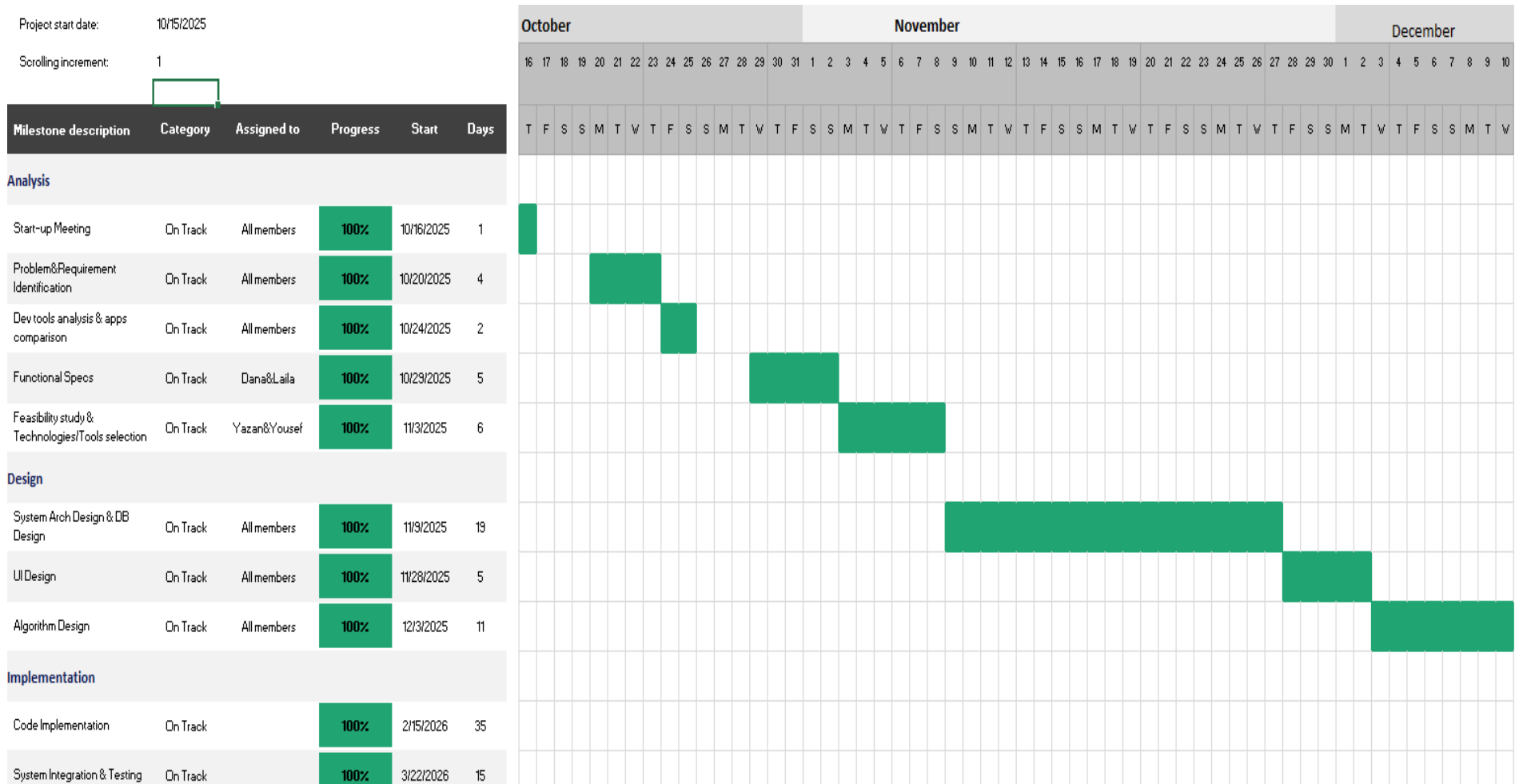


Figure 2.1 Gantt Chart showing the tasks completion and their dependencies for analysis and design phase.

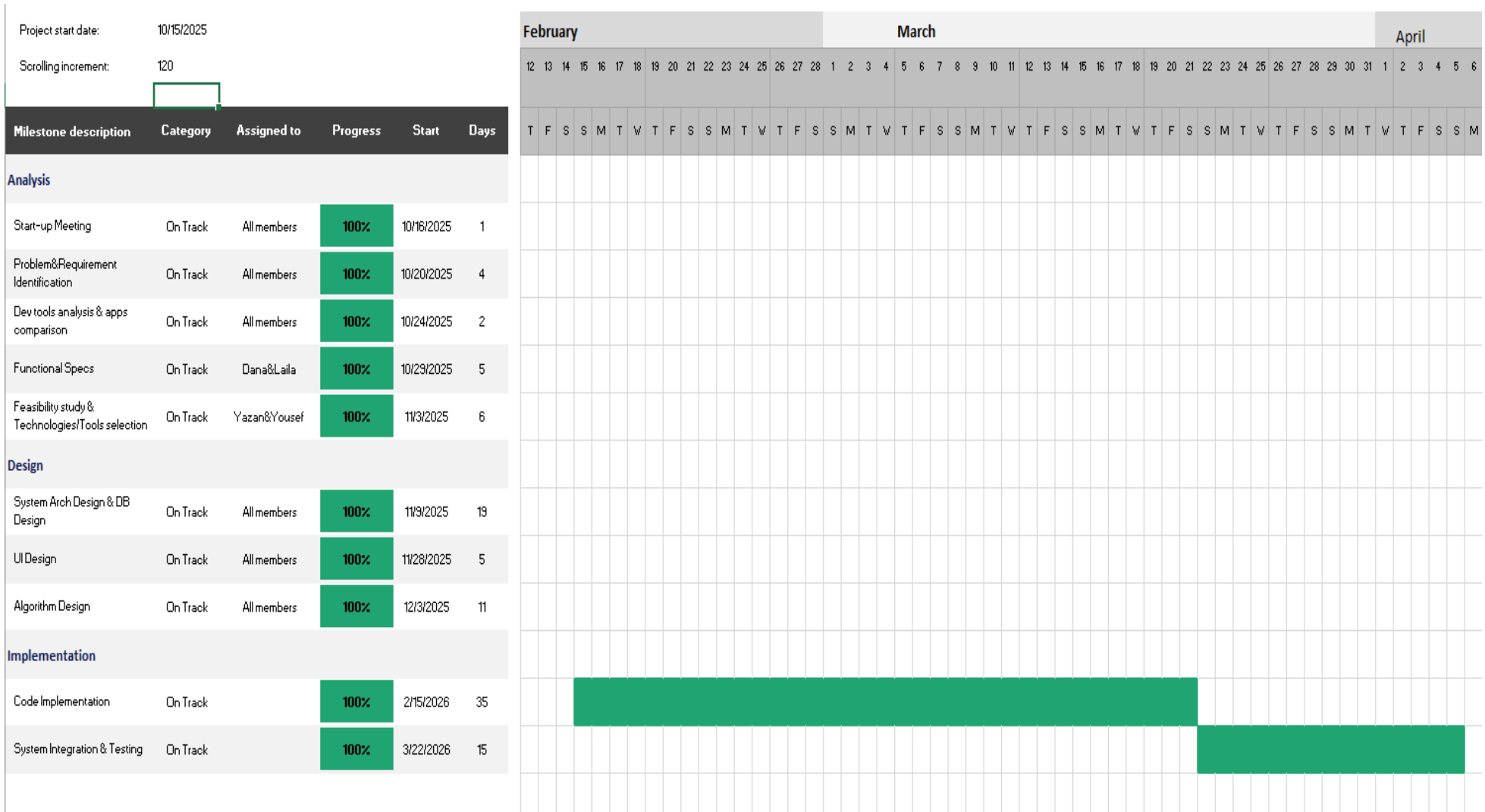


Figure 2.2 Gantt Chart showing the tasks completion and their dependencies for the implementation phase

Roles and Responsibilities

Table 2.3 shows our effectiveness in collaborating with each other and how we distributed the tasks precisely to each one of us. This helped us make a successful project.

Table 2.3 Shows each team member's roles and responsibilities.

Team Member	Role	Responsibilities
Yazan Fani	Documentation, API, Front-end, Back-end	Preparing project documentation, API integration, implementing core functionalities and database integration.
Yousef Saddouq	Documentation, Design, Front-end, Back-end	Preparing project documentation, implementing core functionalities and database integration.
Dana Salem	Documentation, Design, Front-end, Back-end	Preparing project documentation, implementing core functionalities and database integration.
Laila Sadideen	Documentation, Design, Front-end, Back-end	Preparing project documentation, implementing core functionalities and database integration.

Risk Assessment

Table 2.4 shows the risks, probability of this risk occurring, its impact on the system and the mitigation strategy.

Table 2.4 Risk Assessment and Mitigation Plan for the Smart Scheduler System

Risk Description	Probability	Impact	Mitigation Strategy
Not fully understanding system requirements or what students really want because of unclear communication.	Low	Medium	Clarify requirements early by meeting with the supervisor and gathering direct feedback from real students.
Not having enough information about other systems because of limited access.	Medium	Medium	Expand research by checking similar systems, reading papers, and watching demos to fill missing information.
Incompatibility between selected tools or difficulty integrating them later in the project.	Low	High	Test selected tools quickly and create small prototypes to confirm compatibility before full integration.
Database schema may not handle all relationships or might create data redundancy.	Medium	High	Review the database structure with the team and test relationships using sample data to avoid redundancy.
It might be hard to match PSUT's colors and layout while keeping the design simple and user-friendly.	Low	Low	Collect early feedback and adjust the design step by step to match PSUT colors.

Risk Description	Probability	Impact	Mitigation Strategy
The scheduling algorithm may take a long time to generate results.	High	High	Improve the scheduling algorithm gradually by testing each logic part to reduce execution time.
Errors during integration between front-end and back-end modules may cause system crashes.	High	High	Test the modules separately and merge them only after each one works correctly.
Difficulties integrating with the university SIS due to security restrictions or unavailability of the university database.	Medium	High	Design and build a standalone database for the system to allow development and testing even if SIS access is not available.

Cost Estimation

Table 2.5 shows a simple cost estimation for the system, including server hosting, AI-based services, and documentation-related expenses.

Table 2.5 Cost Estimation for the Proposed System

Service	Description	Cost Estimate
Web Hosting (Server)	Hosting the Smart Schedule System on a web server to store and manage scheduling data.	\$300 yearly
AI-Based Services	AI usage for ranking and prioritizing generated schedules based on student preferences.	0.04 Cents per request
Documentation and Printing	Printing final reports and documentation.	\$60

Note: The AI service follows a pay-as-you-go model, with an estimated cost of **\$0.0004 (0.04 cents)** per ranking request. This estimate is based on expected token usage and reflects the lightweight nature of the ranking task, keeping the cost per student very low.

Project Management Tools

Table 2.6 shows lists the tools used by the team throughout the project and their main purposes.

Table 2.6 Shows the tools we used and their description

Tool Name	Description
Google Meet	Used Weekly to meet with group members to discuss the latest updates.
Google Docs	Used to share the documentation across group members.
Google Slides	Used to create the presentation that will be delivered during the project discussion.
EndNote	Used for Citation.
Excel	Used to create the Gantt Chart.
Lucidchart	Used to design UML diagrams.
Draw.io	Used to design UML diagrams.
Miro	Used to design UML diagrams.
Figma	Used to design and visualize the user interface of the system.
GitHub	Used for source code management
DrawSQL	Used to draw the EER Diagram.
PlantUML	Used to draw the class and object Diagram.

Chapter 3 Requirements Specification

Stakeholders

Table 3.1 shows the main stakeholders involved in the system and describes their roles and importance.

Table 3.1 Identified Stakeholders and Their Roles

Stakeholder	Role / Description	Interaction with the System	Importance
Student (End User)	The main user who creates and views personalized schedules.	Adds courses, chooses preferences, and looks at different schedules to compare them.	Main stakeholder; system success depends on meeting student needs.
PSUT SIS System	Manages the backend, database, and access rights.	Updates semester data, tracks performance, and manages integrations	Make sure the system stays stable, updated, and always available.
Academic Advisor	Helps students choose the correct courses according to their study plan.	Look at the chosen schedules and give advice.	Make sure schedules are correct and follow university rules.
Development Team	Builds and maintains the system.	Implements system features, algorithms, and UI.	Keep the system working well and make improvements.
Testing Team	Tests the system to ensure it meets requirements.	Check the system using tools and test cases to ensure the system meets all requirements.	Make sure the system works well without errors.

Platform Requirements

Client-Side Requirements

Table 3.2 describes the requirements that the client (student) must meet in order to have a successful process with the system. Along with its description and whether this requirement is mandatory to have or just recommended.

Table 3.2 Requirements that must be met in the student's perspective, along with its description and status.

Requirement Type	Description	Status
Web Browser	Any modern web browser like Google Chrome, Mozilla Firefox, or Microsoft Edge (latest versions).	Mandatory
Internet Connection	Stable Internet Connection	Mandatory
Device Type	Laptop or Desktop to generate schedules for a better interface.	Recommended

Server-Side Requirements

Table 3.3 describes the requirements that the server must meet in order to have a successful connection with the user. Along with its description and whether this requirement is mandatory to have or just recommended.

Table 3.3 Requirements that must be met in the server's perspective, along with its description and status.

Requirement Type	Description	Status
Internet Connection	Stable Internet Connection	Mandatory
Storage Management	Large Database Storage to handle all students' information.	Mandatory
University SIS API Integration	Needed to retrieve up-to-date data like courses, sections, instructors ...etc.	Mandatory
Web Server	Internet Information Services (IIS)	Mandatory

Functional Requirements

Table 3.4 shows the functional requirements. These functional requirements define the core functionalities of the scheduling system by describing the actions performed by the user and the system to create and manage schedules.

Table 3.4 Detailed Functional Requirements Including Inputs, Processes, Outputs, and Priorities

ID	Requirement Description	Input	Process	Output	Priority
FR-1	User Login & Authentication	Username, Password	Validate credentials with university SIS	Grant or deny access	Essential
FR-2	Retrieve Remaining Courses	Student ID	Fetch incompletd, allowed courses	Display list of remaining courses	Essential
FR-3	Add Course to "Added Courses" List	Selected course(s)	Store user selection and update added-course list	Updated added-courses schedule	Optional
FR-4	Select Instructor Preference	Dropdown menu of instructors for each added course	Filter all course sections based on the selected instructor(s)	Display only sections taught by selected instructor(s)	Optional
FR-5	Set Time Range	Minimum time & Maximum time	The system removes any class that does not fall between the time range the student chose.	Only classes within the chosen time range	Optional

ID	Requirement Description	Input	Process	Output	Priority
FR-6	Set Break Time	Minimum break time & Maximum break time	Keep only the classes that have breaks within the chosen minimum and maximum time	Schedules that follow the break-time limits	Optional
FR-7	Select Preferred Attendance Days	List of preferred days	Filter out any schedules that do not match the selected days	Schedules that follow the student's chosen attendance days	Optional
FR-8	Set Credit Hours	Selected number of credit hours	Filter out schedules that do not match the selected credit hours	Schedules that match the student's chosen credit hours.	Essential
FR-9	Generate Non-Conflicting Schedules	Added courses + filters	Create non-overlapping schedules from the added courses and filters	List of generated schedules	Essential
FR-10	Rank Generated Schedules	Generated schedules, student grades, course importance, selected filters	Calculate a ranking score for each generated schedule based on course importance, student grades, and match with selected filters	Ranked list of schedules	Optional

ID	Requirement Description	Input	Process	Output	Priority
FR-11	Mark Schedule as Favorite	Selected schedule	Save schedule in “Favorites” list	Favorited schedule saved for quick access	Optional
FR-12	Export Schedule (PDF)	Selected schedule	Convert schedule to exportable format	Downloadable file	Optional

Non-Functional Requirements

Table 3.5 shows non-functional requirements. These non-functional requirements define the standards of the scheduling system by showing the performance, security, reliability, etc... Meeting them makes the system secure, fast, reliable, and easy to use.

Table 3.5 Non-Functional Requirements of the system with their Descriptions and Examples

Category	Requirement Description	Example
Performance	The system must generate all possible schedules within 2 seconds after the user clicks on the generate button.	The user select courses and filters, then clicks generate. The system displays the generated schedules in under 2 seconds.
Security	The system must provide security and protect the user data from any unauthorized access.	HTTPS on the user’s browser.
Category	Requirement Description	Example

Reliability	The system must be available 24/7 as well as during registration periods with an uptime of at least 99.5%.	Students can access the system continuously when the registration periods are on doors without system crashes.
Scalability	The system must support at least 50% increase in the number of concurrent users during peak registration times.	If 3,000 students use the system normally, it should still run efficiently with 6,000 students.
Usability	Students must be able to generate a schedule in no more than three main steps.	Students add courses, select filters/preferences, then clicks generate.

Other Requirements

Table 3.6 shows additional system requirements that support security, data handling, and compliance

Table 3.6 Additional System Requirements Beyond Functional and Non-Functional Specifications

Requirement Type	Description	Example
API Usage Restrictions	Only RESTful APIs using JSON format are allowed.	Integration with external API.
Data Transmission Protocol	All communications must use secure HTTPS/TLS 1.3.	All the data transferred between the client and the server is encrypted using HTTPS.
Data Storage Format	Data must be stored in UTF-8 encoded data.	Log files saved in standardized format.
Compliance	The system must comply with GDPR and local data protection laws.	User consent required for data collection.

Chapter 4 System Design

Logical Model Design

4.1.1 Architecture Overview

Table 4.1 shows a high-level overview of the system architecture, including its main components and technologies.

Table 4.1 High-Level Architecture Overview of System Layers and Technologies

Layer / Component	Responsibilities	Tech Choices	Interfaces
Client (Web)	Presentation, input validation, UI state	React	REST
API Gateway / Backend	Business logic, authentication, preference handling, schedule orchestration	.NET Core	HTTPS/JSON
Scheduling Engine	Apply filters, detect conflicts, generate and rank schedules	Internal algorithm	Internal APIs
Database	Persistent storage for courses, sections, instructors, students, filters, and schedules	Oracle Database	SQL
External Integration	Synchronization with university registration and academic data	PSUT SIS	Secure REST APIs

4.1.2 Required UML Diagrams

1. Architecture Diagram

Figure 4.1 shows the architecture diagram of our proposed system. The API first sends a request to the Back-end service (.NET Core), which in turn fetches the courses data from the database, then the student chooses the preferred courses and filters. Student's selected courses and filters are stored in our database. After that, the scheduling engine, which is our optimization algorithm, generates non-conflict schedules based on the selected filters. The data stores layer (Database Container) represents the tables in the database that store information like courses, sections, instructors, students, filters, and generated schedules. This whole process is integrated with the university registration system.

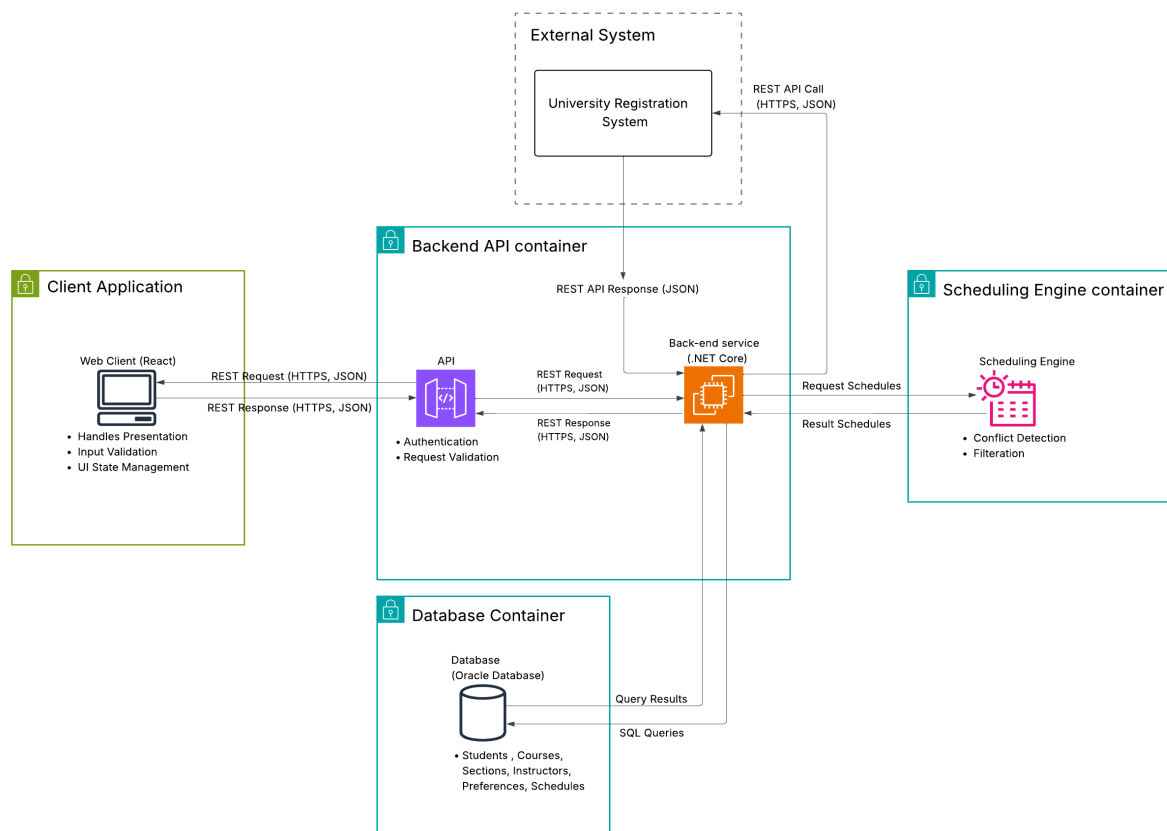


Figure 4.1 Architecture Diagram showing the components of the system and how they're related.

2. Class Diagram

Figure 4.2 shows the main components of the system as classes with attributes and methods. It also shows the association relationships between these classes and how they are related.

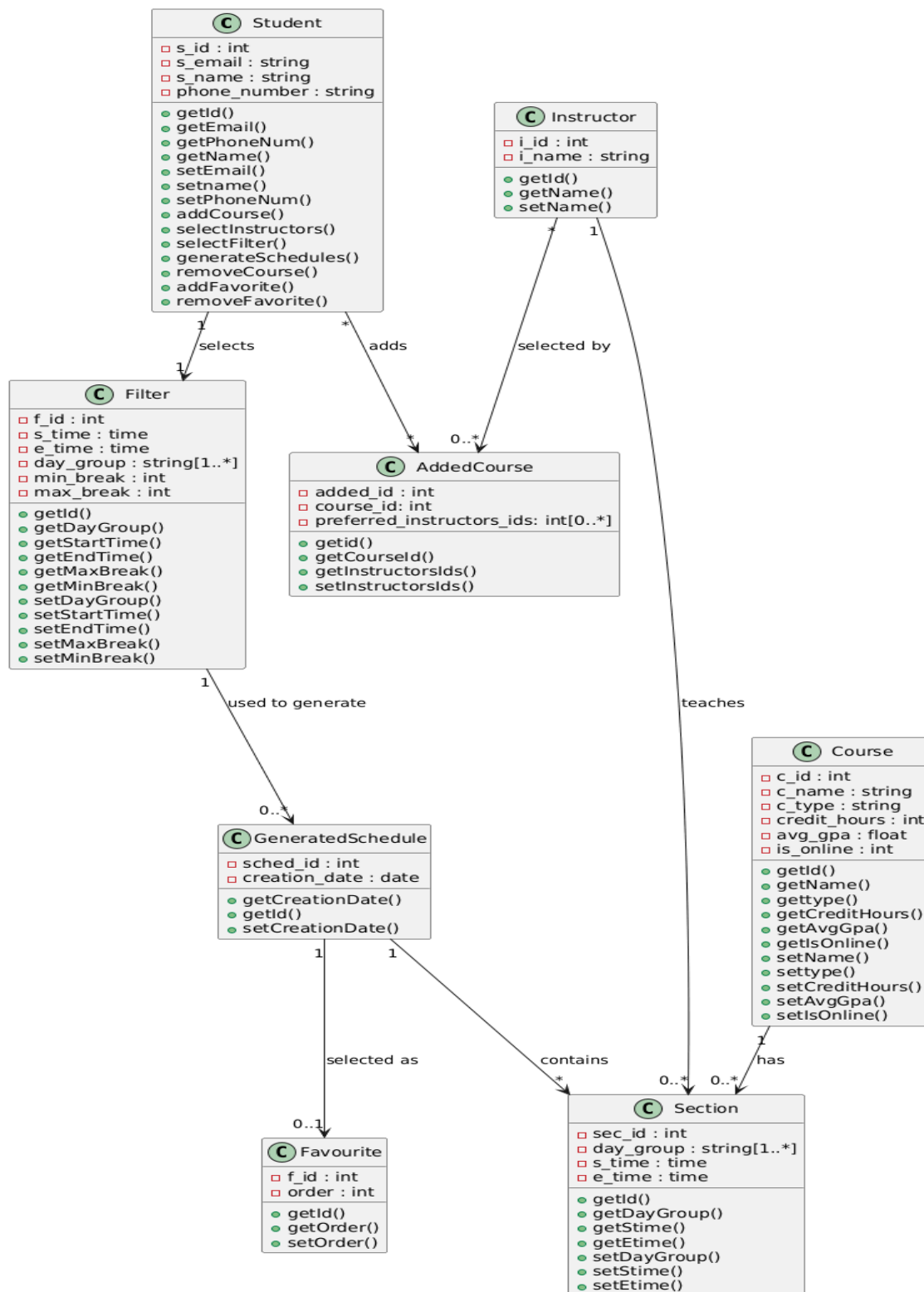


Figure 4.2 Class Diagram showing the main components of the system as classes with attributes and methods.

3. Object Diagram

Figure 4.3 shows instances for the student, instructor, course, section, and added course classes. It also shows the relationships between them and the process of a student selecting the courses he wants.

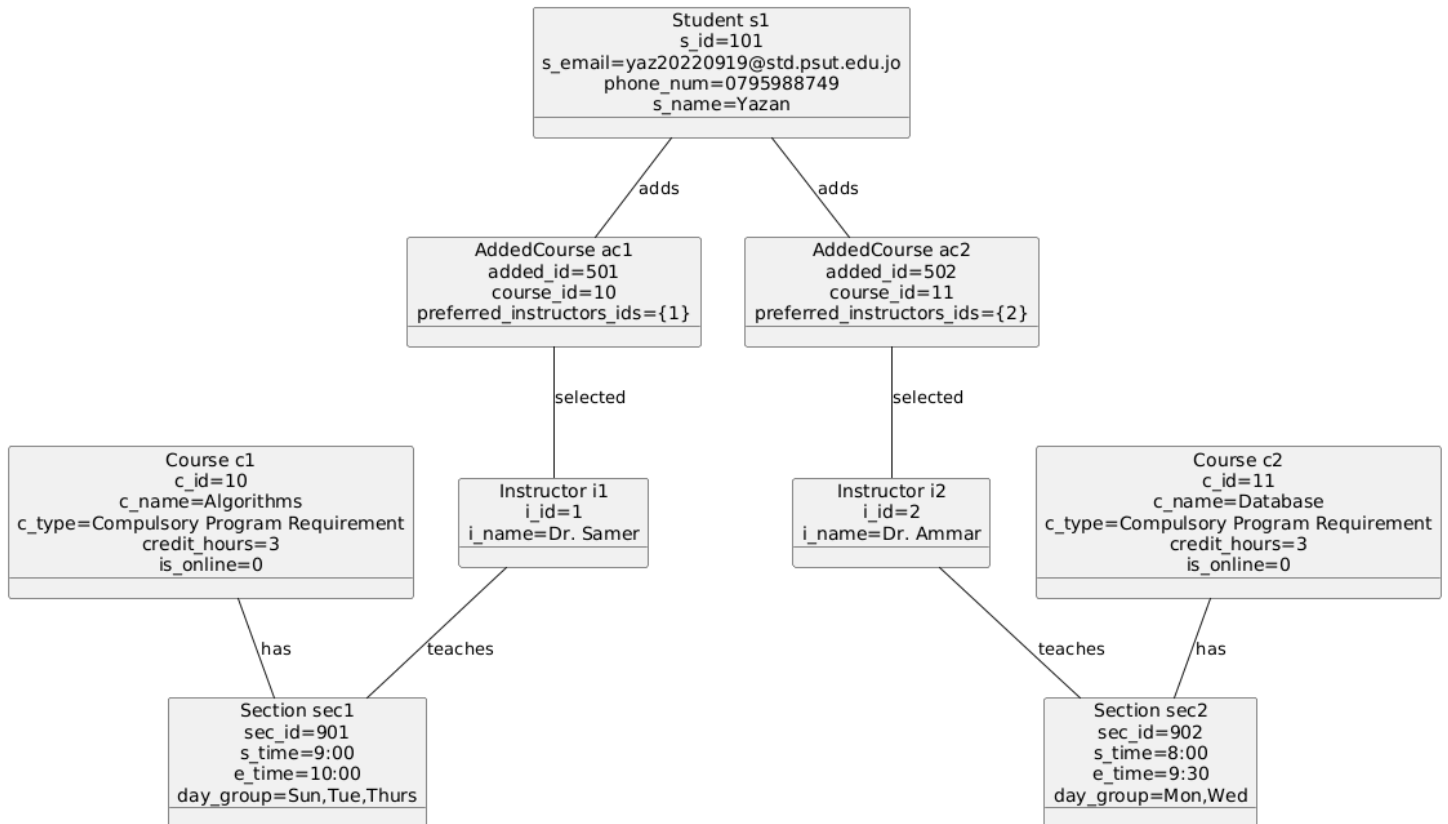


Figure 4.3 Object Diagram showing instances for the student, instructor, course, section, and added course classes.

Figure 4.4 shows instances of the student, filter, generated schedule, and section classes, and the process of selecting filters to generate schedules.

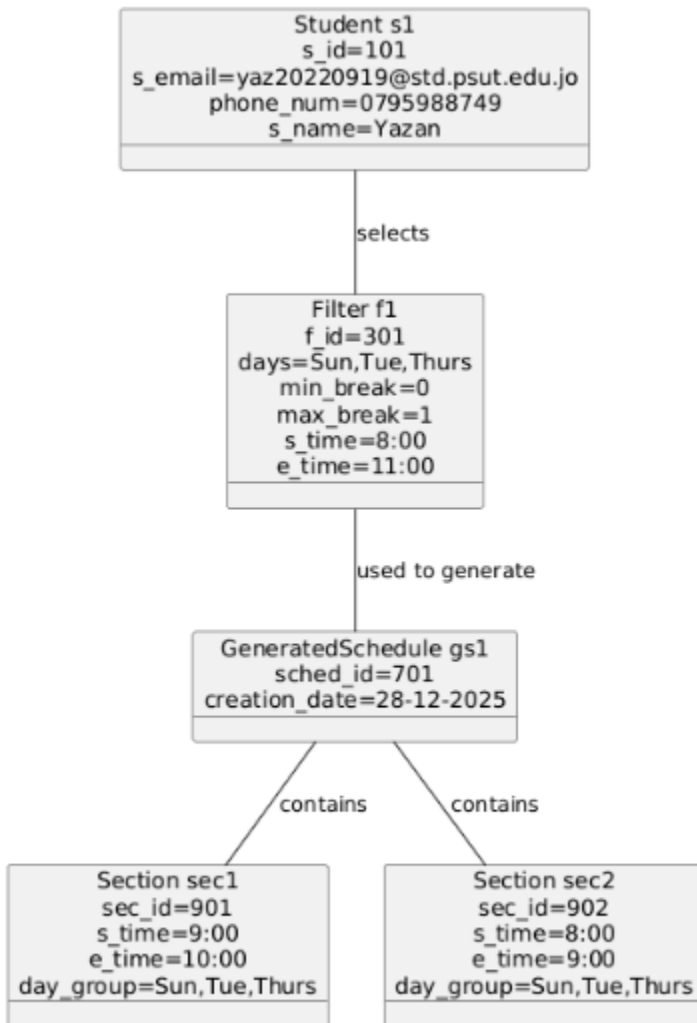


Figure 4.4 Object Diagram showing instances of the student, filter, generated schedule, and section classes.

Figure 4.5 shows an instance of the generated schedule class and an instance of the favorite class, along with the relationship between them when a schedule is selected as favorite.

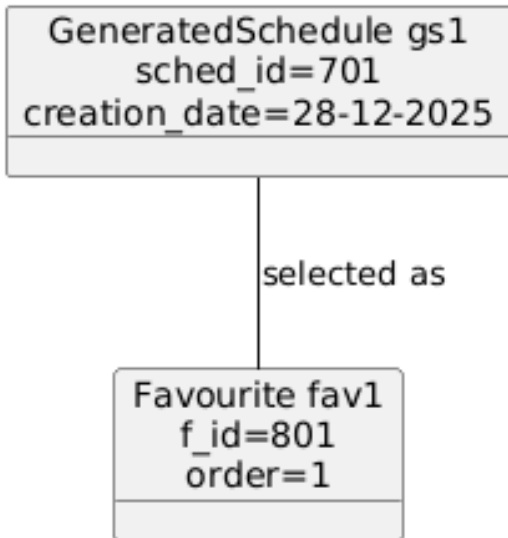


Figure 4.5 Object Diagram showing an instance of the generated schedule class and favorite class.

4. Use Case Diagram

Figure 4.6 shows the main actions that the student can perform in the Smart Scheduling System, starting from viewing courses and ending with generating and managing schedules.

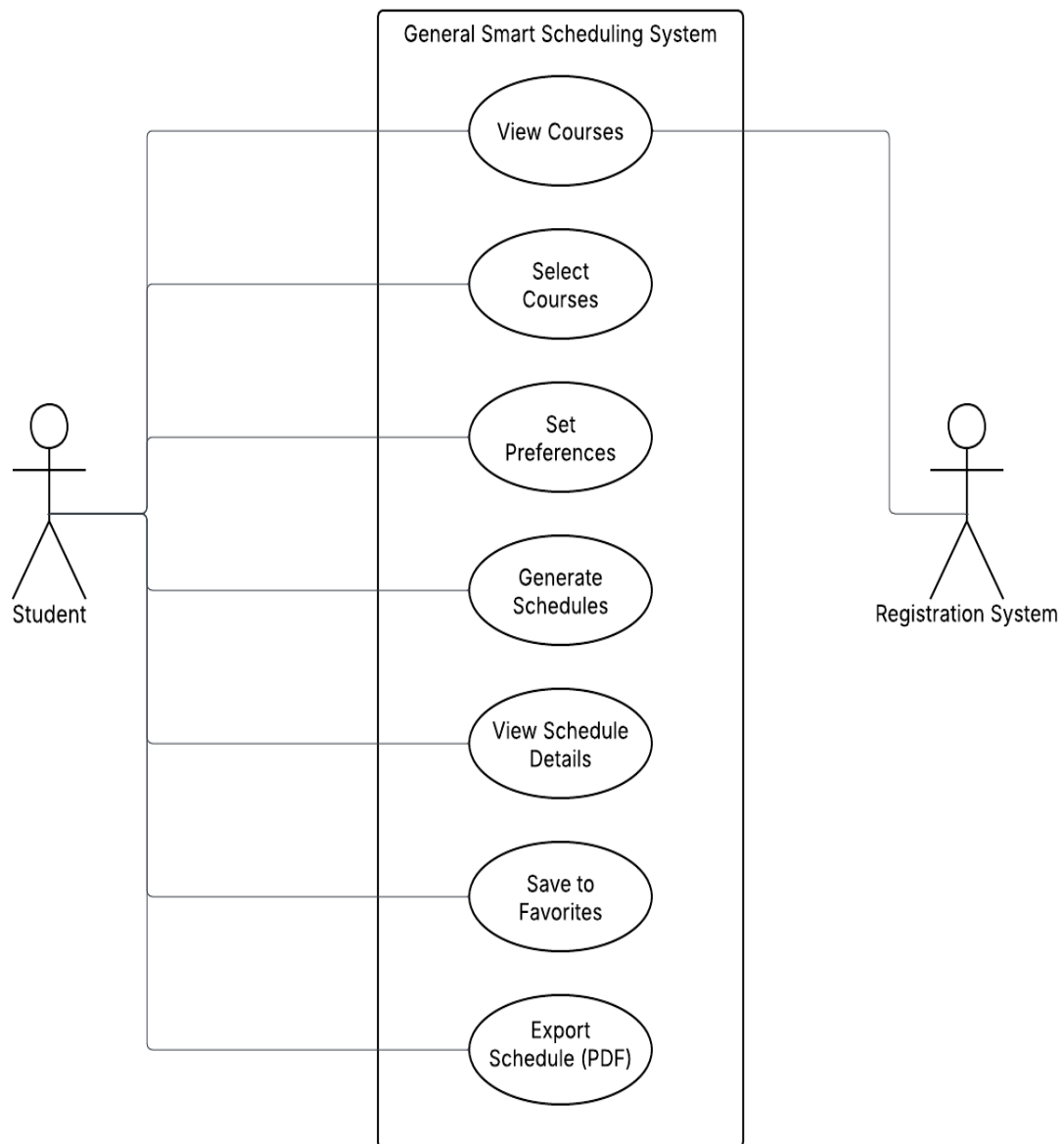


Figure 4.6 Use Case Diagram showing the student's main interactions with the Smart Scheduling System

- **Use Case:** Generate Schedules
- **Actors:** Student
- **Purpose:** The purpose of this use case is to help the student create suitable class schedules for the semester. The system aims to generate schedules that do not have time conflicts and match the student's selected courses and preferences.
- **Overview:** In this use case, the student starts by choosing the courses they want to take during the semester. After that, the student can set some preferences. Once everything is ready, the student asks the system to generate schedules. The system then processes the student's choices and creates several schedules that do not contain conflicts. These schedules are displayed to the student so they can review them.
- **Type:** Primary
- **Cross References:** FR-9
- **Typical Course of Actions:**

Table 4.2 Typical Course of Actions for the Generate Schedules Use Case

Actor Action	System Response
1. The student opens the Smart Scheduler system.	2. The system displays the list of available courses.
3. The student selects the courses he/she wants to register for.	4. The system updates the added courses list.
5. The student sets scheduling preferences.	6. The system saves the chosen preferences.
7. The student clicks the "Generate Schedules" button.	8. The system collects the selected courses and preferences.
—	9. The system processes the data and runs the scheduling logic.

Actor Action	System Response
—	10. The system generates a list of conflict-free schedules based on the student's preferences and displays them on the screen.
11. The student reviews the generated schedules.	—

Figure 4.7 focuses on how the student sets schedule preferences such as days, time range, break time, and credit hours.

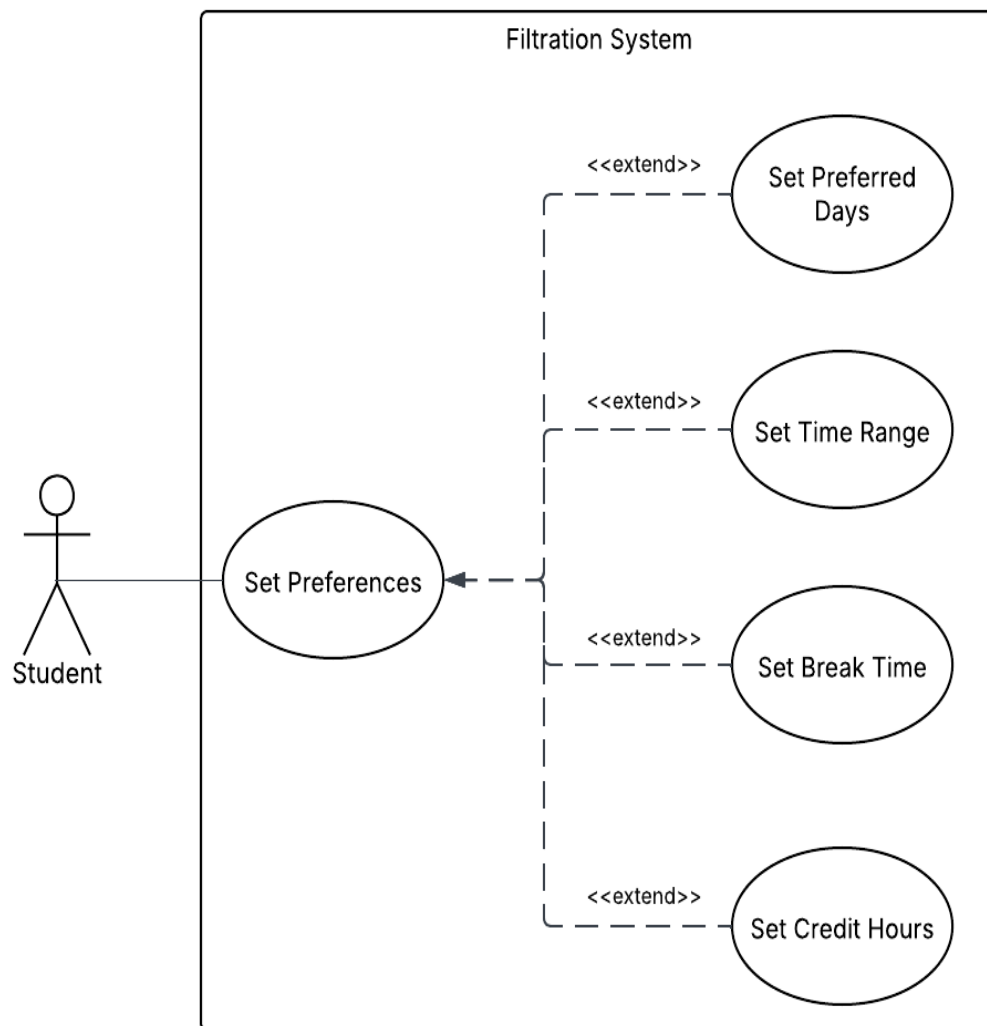


Figure 4.7 Use Case Diagram illustrating how the student configures schedule preferences.

- **Use Case:** Set Preferences
- **Actor:** Student
- **Description:** The student sets the basic preferences for the schedule, such as preferred days, suitable class times, break time between lectures, and the desired number of credit hours. After the student finishes, the system saves these preferences and uses them later when generating schedules.

Figure 4.8 shows what the student can do after schedules are generated, such as viewing schedule details, saving schedules, and exporting them.

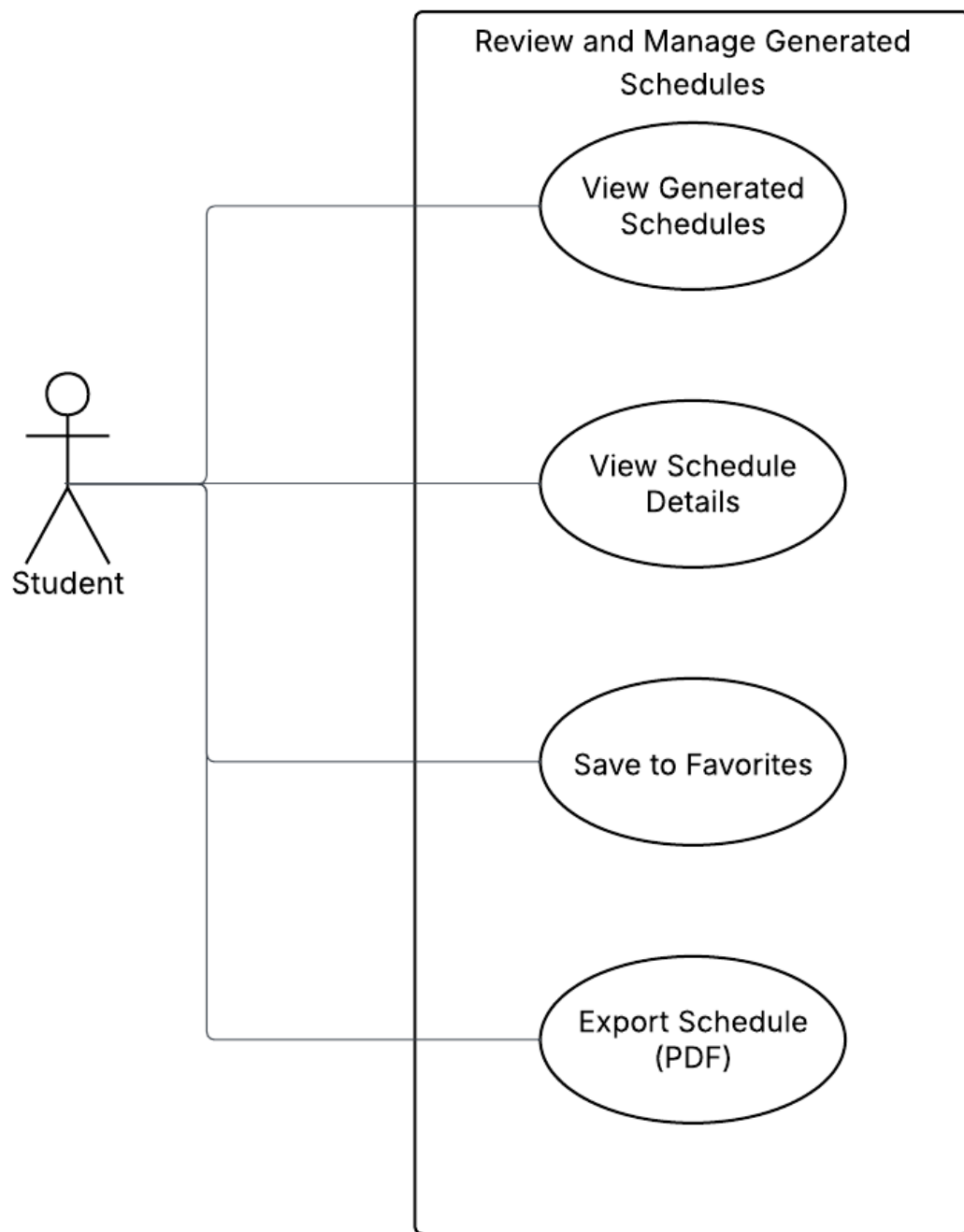


Figure 4.8 Use Case Diagram showing how the student reviews and manages generated schedules.

- **Use Case:** View Generated Schedules
- **Actors:** Student
- **Purpose:** Help the student view the schedules generated by the system and start reviewing the results.
- **Overview:** After the student clicks the generate button, the system shows a list of available non-conflicting schedules. The student can browse the list and choose any schedule to see it clearly and decide what to do next (such as saving it or exporting it).
- **Type:** Primary
- **Cross References:** FR-9
- **Typical Course of Actions:**

Table 4.3 Typical Course of Actions for the View Generated Schedules Use Case

Actor Action	System Response
—	1. The system displays valid schedules and provides navigation options to move between schedules.
2. The student browses the schedules using the Next and Previous buttons and views the details of each generated schedule.	—
3. The student may choose to export the schedule as a PDF and/or mark it as a favorite.	4. The system exports the PDF file and/or saves the schedule to the favorites list.

5. Activity Diagram

Figure 4.9 shows the process of logging in and checking credentials by the PSUT SIS. Based on the verification result, the user is either granted access or asked to try again.

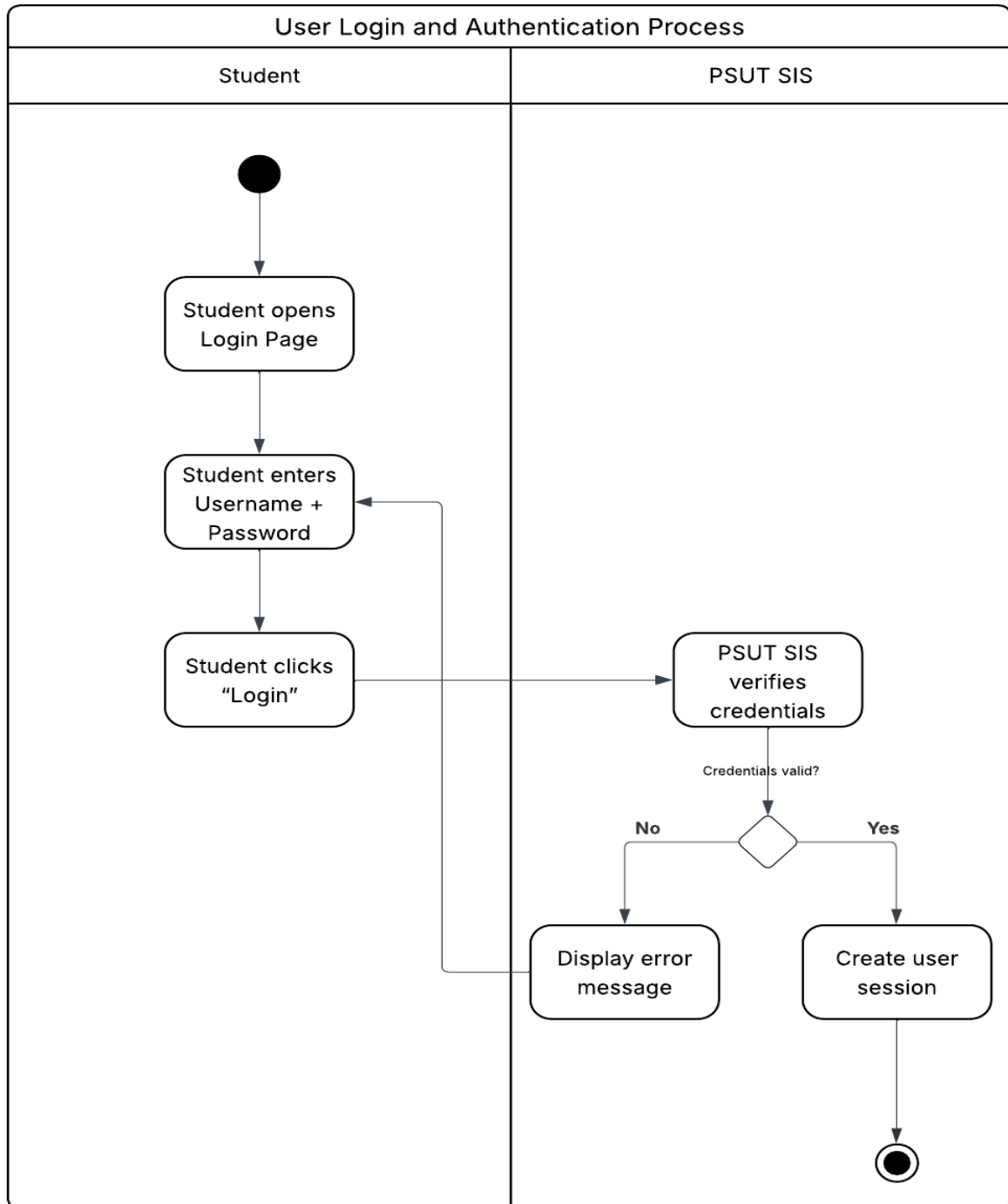


Figure 4.9 Activity Diagram for User login and authentication process

Figure 4.10 shows what happens when the student enters the Smart Scheduling System. The system gets the student data and available courses from the PSUT SIS and then shows the main schedule page with the available options.

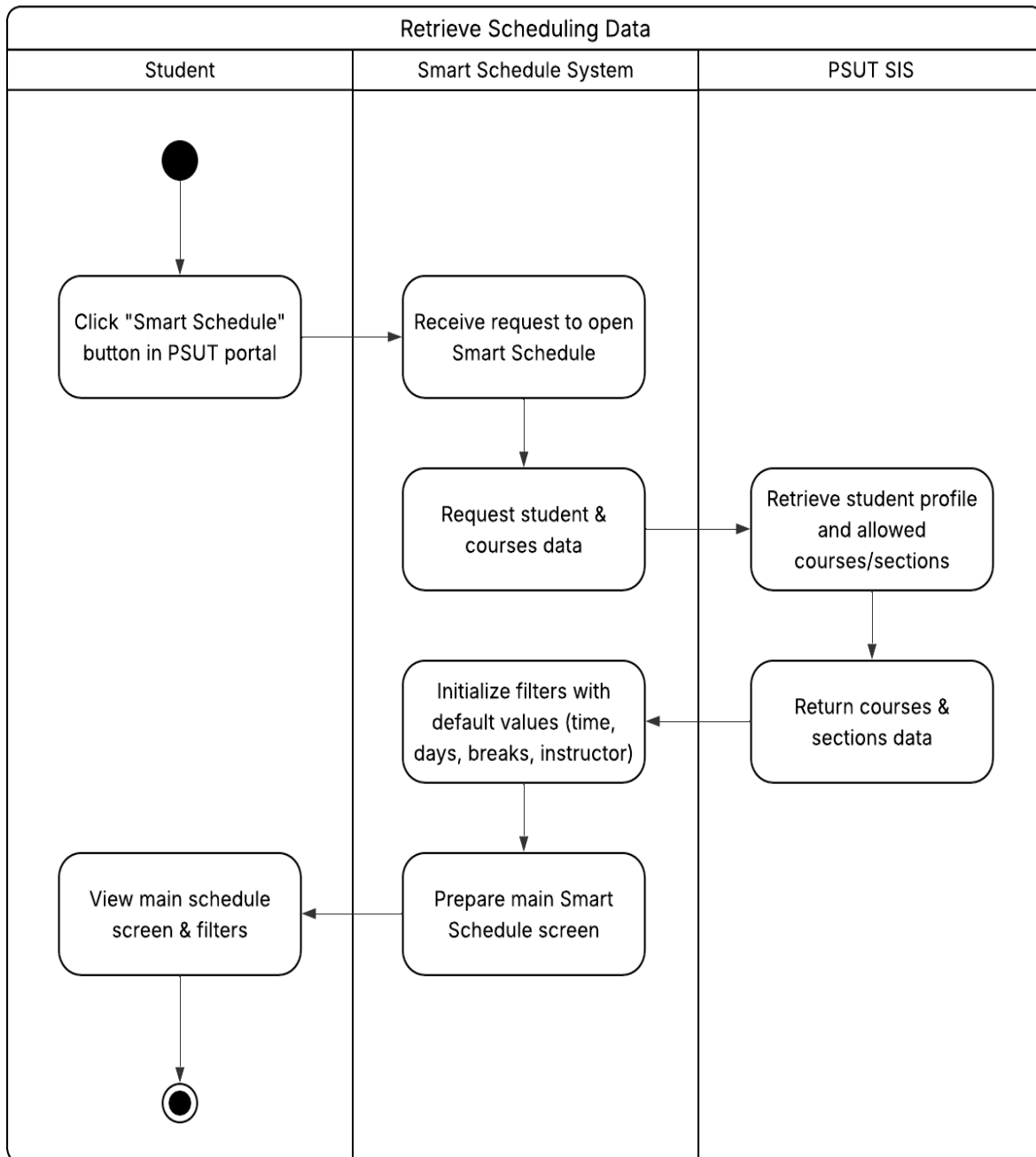


Figure 4.10 Activity Diagram for Opening the Smart Schedule and Loading Initial Data

Figure 4.11 explains how the student chooses courses and sets preferences. After that, the student asks the system to generate schedules. The system processes the information and either shows suitable schedules or informs the student if no schedules are found.

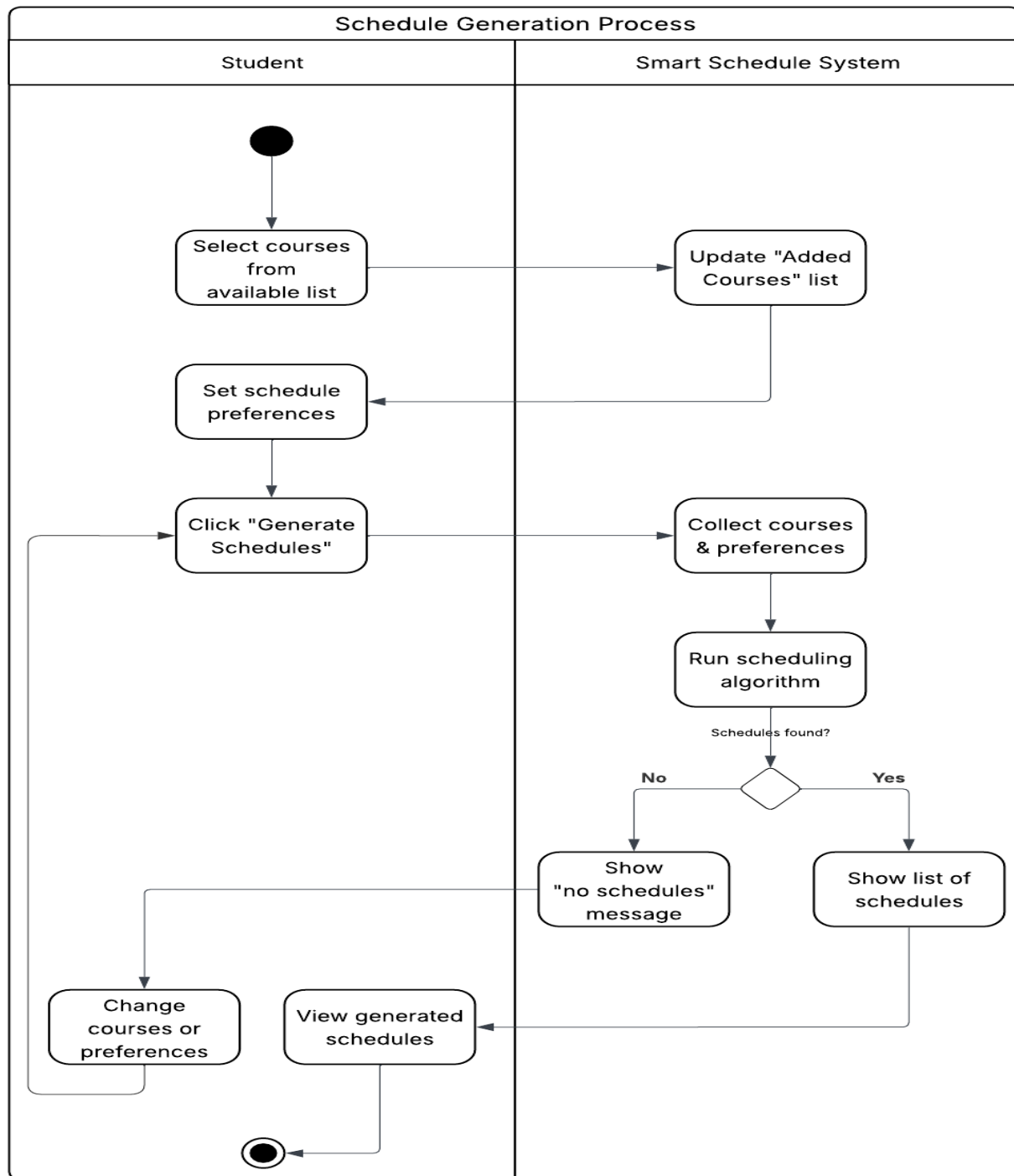


Figure 4.11 Activity Diagram for Selecting Courses and Generating Schedules

Figure 4.12 shows what the student does after schedules are generated. The student can view the schedules, check their details, save a schedule to favorites, or export it as a PDF file.

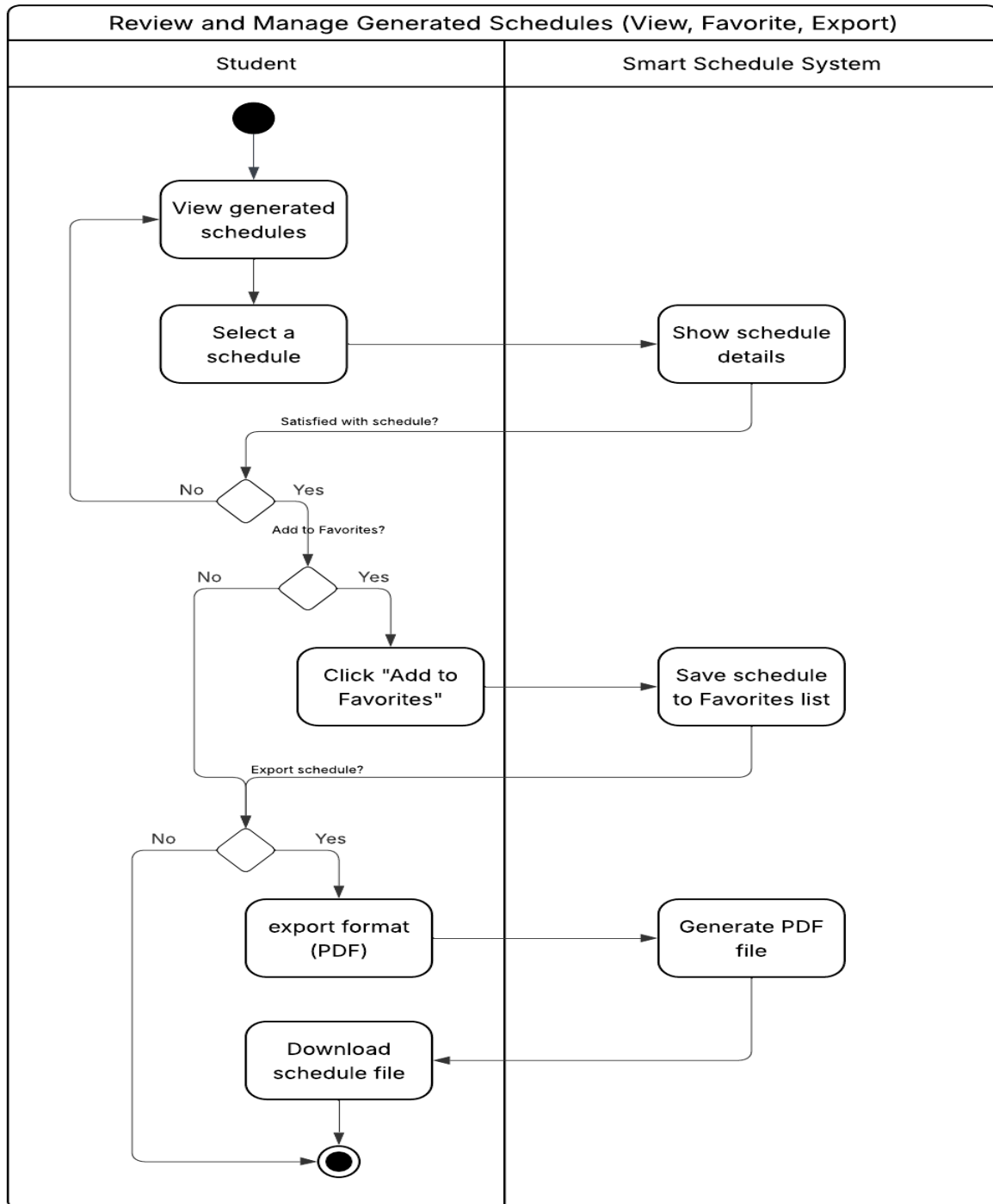


Figure 4.12 Activity Diagram for Reviewing, Favoring, and Exporting Schedules

6. Sequence Diagram

Figure 4.13 shows how the system requests data from the API to display the courses' information into the screen. The student can add courses from the displayed data.

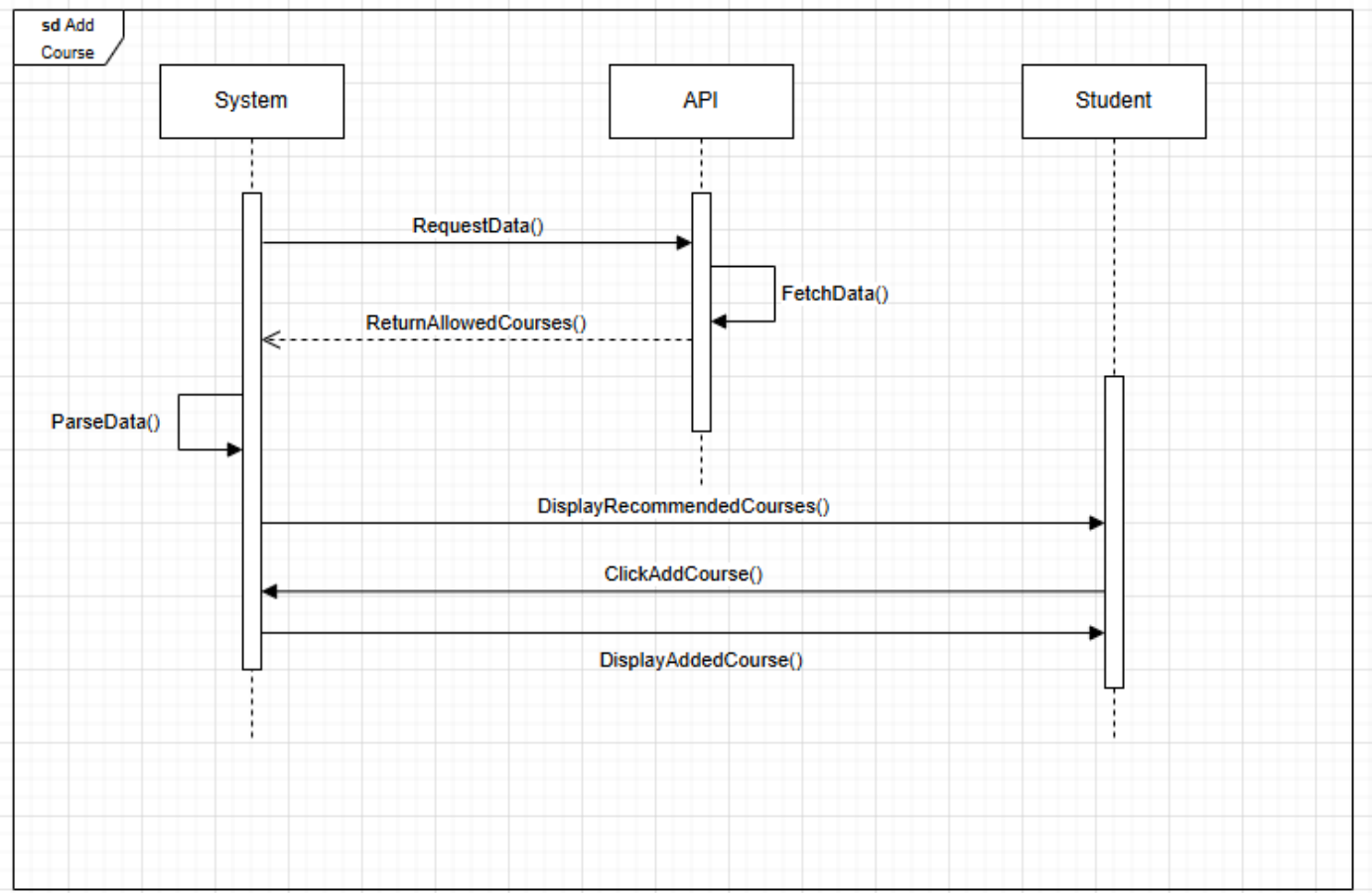


Figure 4.13 Sequence Diagram showing the Add course scenario.

Figure 4.14 shows how the student can remove any course added by mistake or if he changed his mind.

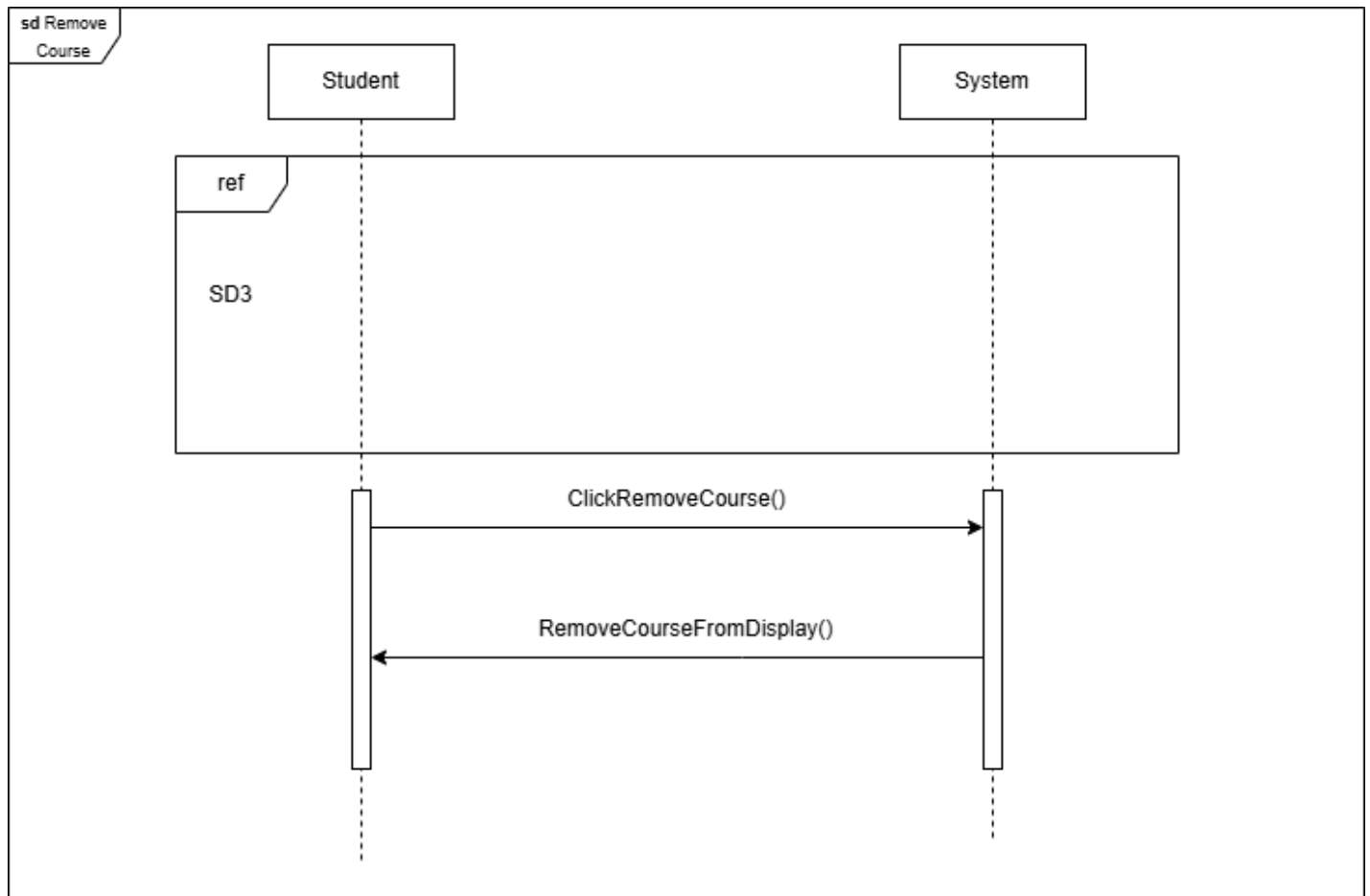


Figure 4.14 Sequence Diagram showing the Remove Course Scenario.

Figure 4.15 shows how the student adds the required filters. Then clicks 'generate' to generate all schedules if there are any, if the filters selected were too strict that no schedule is possible, no output message will be displayed.

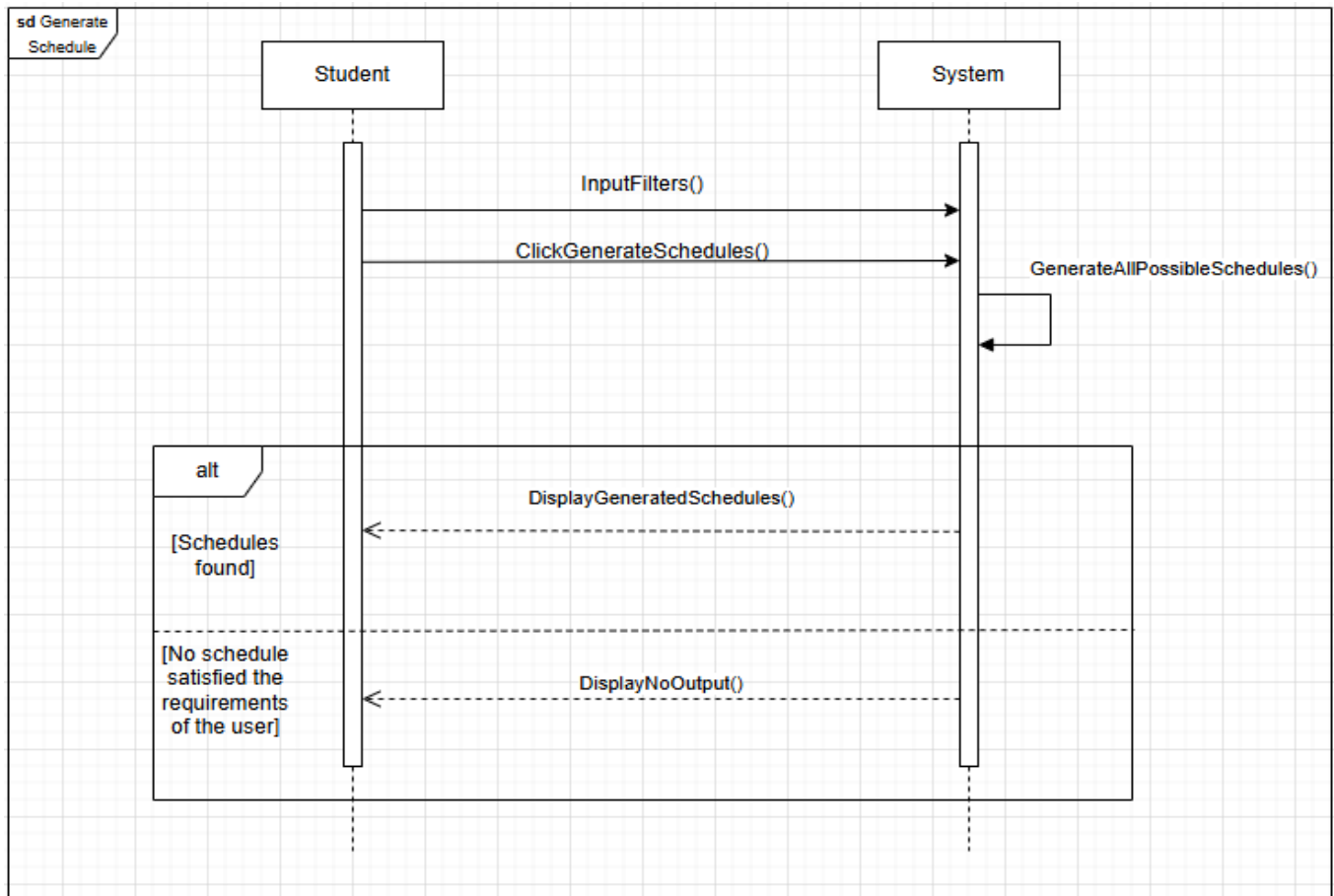


Figure 4.15 Sequence Diagram showing how the student can generate schedules.

Figure 4.16 shows the student login process. The user enters his information, if they're correct then the student is logged in successfully, else an error message will appear.

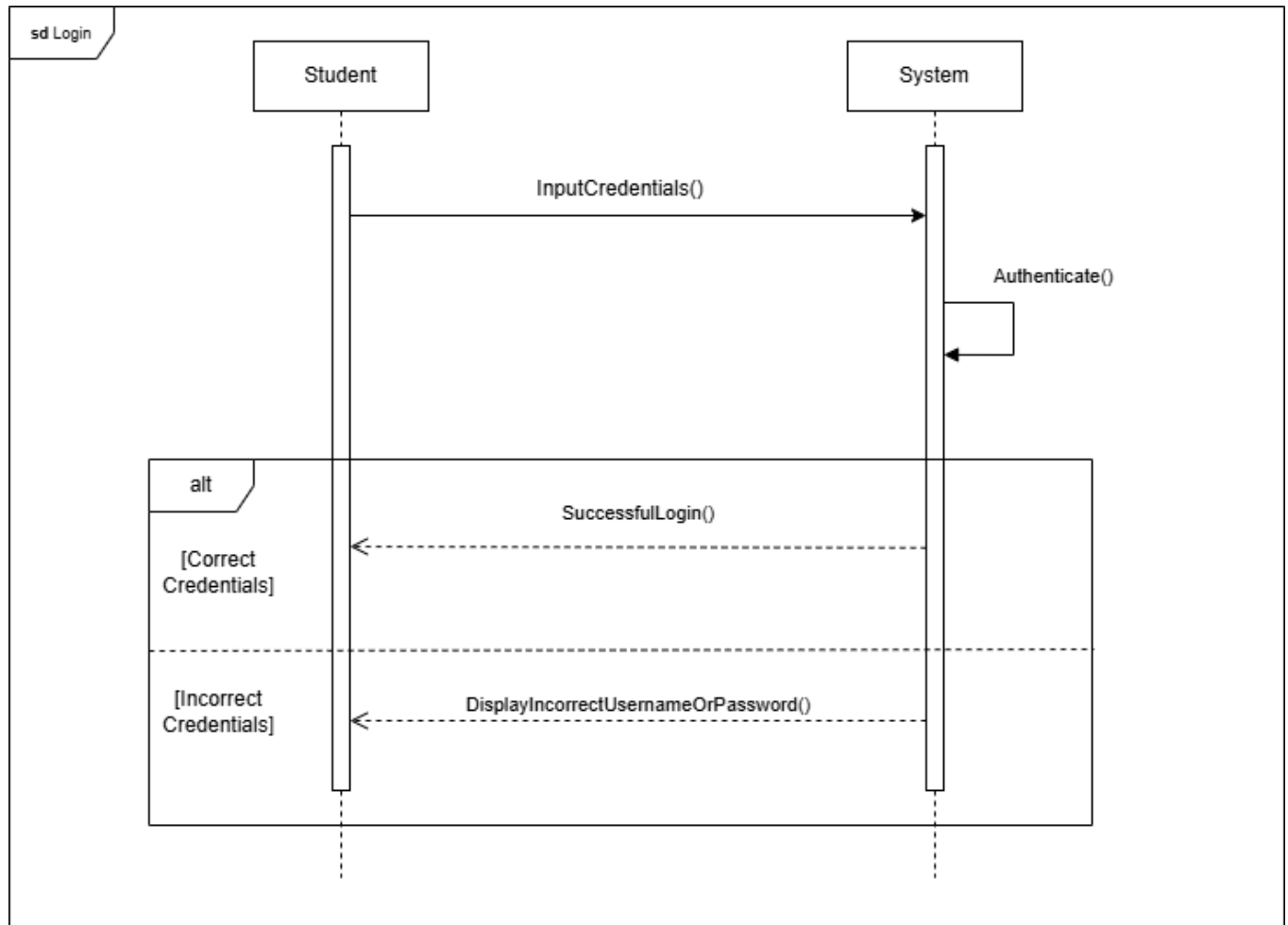


Figure 4.16 Sequence diagram showing the login process.

7. State Transition Diagram

Figure 4.17 shows that a course starts as Not Selected, becomes Selected when added, and returns to Not Selected if removed.

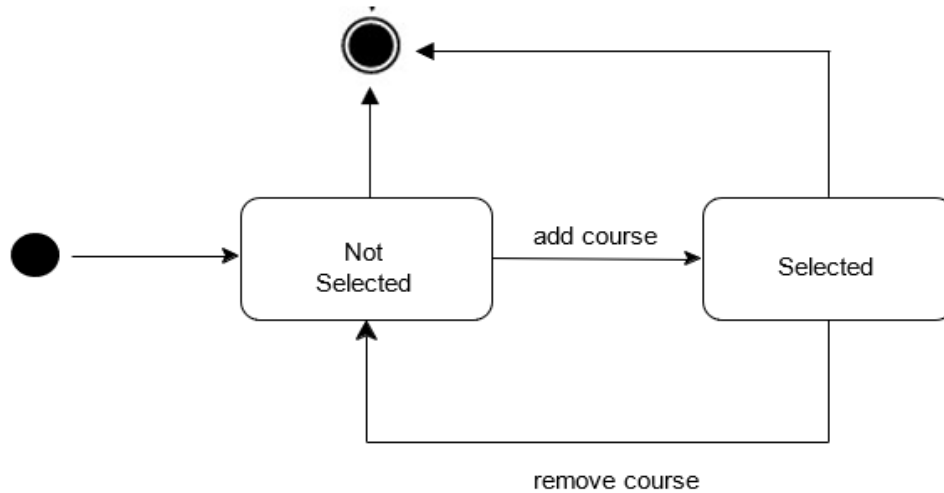


Figure 4.17 State Diagram showing how the student selects and removes courses.

Figure 4.18 explains how filters are used in the system. The user can continue with default filter values, select some filters while others remain default, or choose all available filters. The filter state updates when filters are changed or removed.

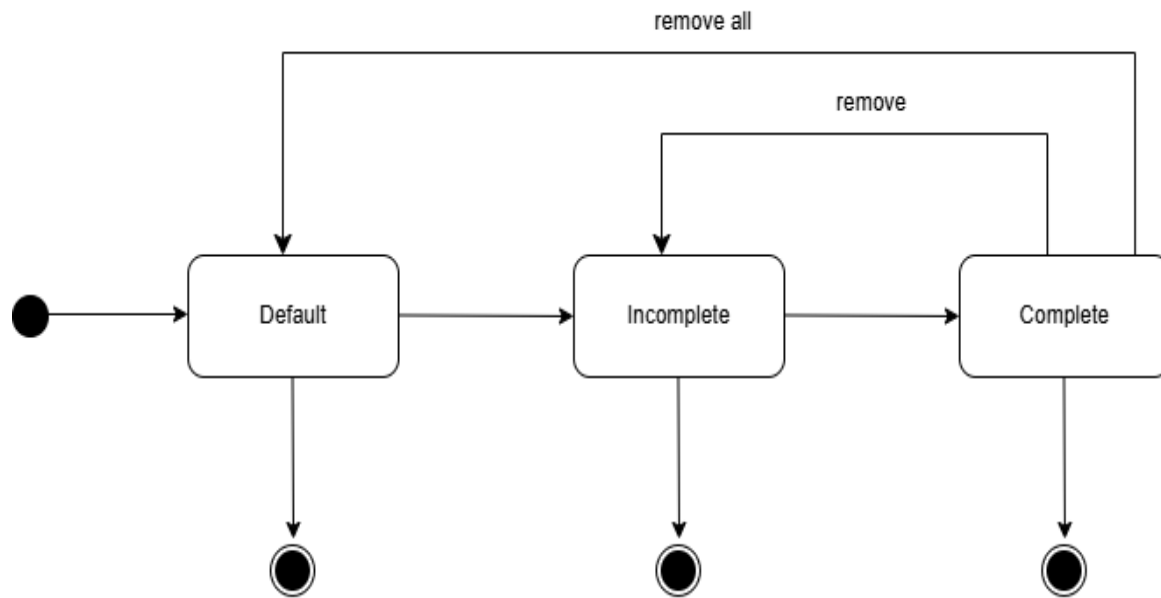


Figure 4.18 State Diagram showing the different filter states within the system.

Figure 4.19 shows an internal process. It starts in the Empty state. After the user selects filters and clicks Generate, the system moves to in Progress, where schedules are built. Finally, it reaches Complete, where multiple schedules are generated.

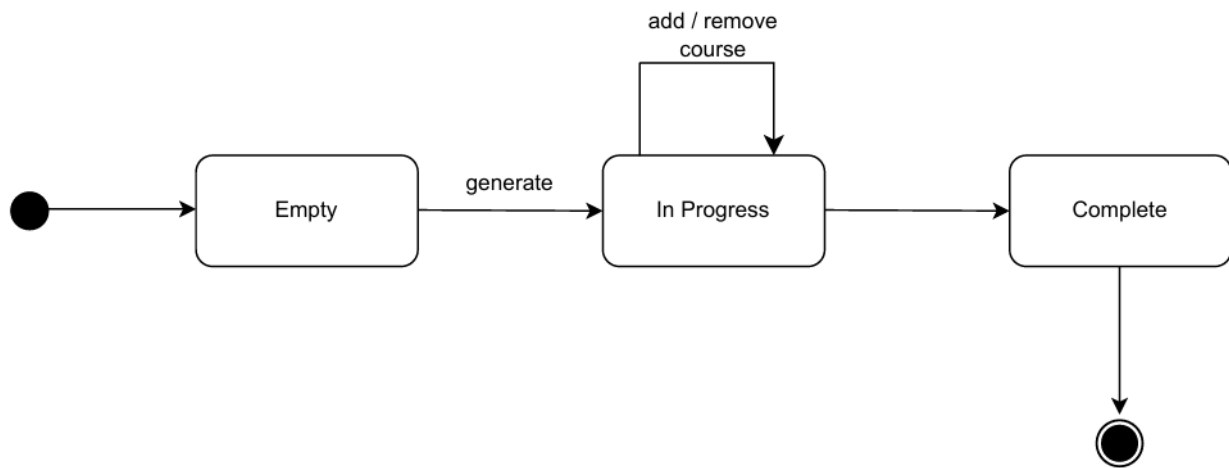


Figure 4.19 State Diagram showing the schedule generation process in the system.

4.1.3 Data Design – ERD & Database Schema

Table 4.4 shows the database tables' details. Each table column with its type, is it a primary key or foreign, is it nullable or not, and constraints on the column.

Table 4.4 Database tables, and information about their keys.

Table	Column	Type	PK/FK	Nullable	Notes/Constraints
Student	St_id	INT	PK	No	Student's university id.
Student	St_email	VARCHAR(255)	-	No	UNIQUE
Student	Phone_num	VARCHAR(20)	-	Yes	-
Course	C_id	INT	PK	No	Course number as in the university portal.
Course	C_name	VARCHAR(255)	-	No	-
Course	C_hrs	INT	-	No	-
Course	Avg_gpa	DECIMAL(3,2)	-	Yes	-
Course	Is_online	BOOLEAN	-	No	-
Course	C_type	VARCHAR(50)	-	No	-
Student_course	S_id	INT	PK,FK→ student.St_id	No	ON DELETE CASCADE
Student_course	C_id	INT	PK,FK→ course.C_id	No	ON DELETE CASCADE

Table	Column	Type	PK/FK	Nullable	Notes/Constraints
Instructor	I_id	INT	PK	No	Auto-generated
Instructor	I_name	VARCHAR(255)	-	No	-
Section	Sec_id	INT	PK	No	Auto-generated
Section	S_time	TIME	-	No	-
Section	F_time	TIME	-	No	-
Section	C_id	INT	PK, FK→ course.C_id	No	-
Section	I_id	INT	FK→ instructor.I_id	No	-
Instructor_added	I_id	INT	PK,FK→ instructor.I_id	No	-
Instructor_added	C_id	INT	PK,FK→ Student_Course.C_id	No	-
Instructor_added	St_id	INT	PK,FK→ Student_Course.St_id	No	-
Day_group_filter	F_id	INT	PK	No	Auto-generated
Day_group_filter	Day	VARCHAR(20)	PK	No	Sun, Mon, Tue,...
Filter	F_id	INT	PK	No	Auto-generated
Filter	S_time	TIME	-	Yes	-

Table	Column	Type	PK/FK	Nullable	Notes/Constraints
Filter	F_time	TIME	-	Yes	-
Filter	Min_break	INT	-	Yes	In hours
Filter	Max_break	INT	-	Yes	In hours
Filter	St_id	INT	FK→ Student.St_id. C_id	Yes	-
Generated_Schedule	Sched_id	INT	PK	No	Auto-generated
Generated_Schedule	Creation_date	DATE	-	No	-
Generated_Schedule	F_id	INT	PK, FK→ Filter.F_id	No	-
Generated_Section	Sec_id	INT	PK,FK→ Section.Sec_id	No	-
Generated_Section	Sched_id	INT	PK,FK→ Generated_Schedule.Sched_id	No	-
Generated_Section	C_id	INT	PK,FK→ Section.C_id	No	Composite PK
Day_Group_Section	Sec_Id	INT	PK, FK→Section.Sec_id	No	-
Day_Group_Section	Day	VARCHAR(20)	PK	No	-
Day_Group_Section	C_id	INT	PK, FK→Section.C_id	No	-

Table	Column	Type	PK/FK	Nullable	Notes/Constraints
Favourite	Fav_id	INT	PK	No	Auto-generated
Favourite	Order	INT	-	No	Ranking/Priority
Favourite	Sched_id	INT	PK, FK→ Generated_Schedule.Sched_id	No	-
Favourite	F_id	INT	PK, FK→ Generated_Schedule.F_id	No	-

Figure 4.20 shows the entities of our system. Each entity along with its attributes and its relation with other entities.

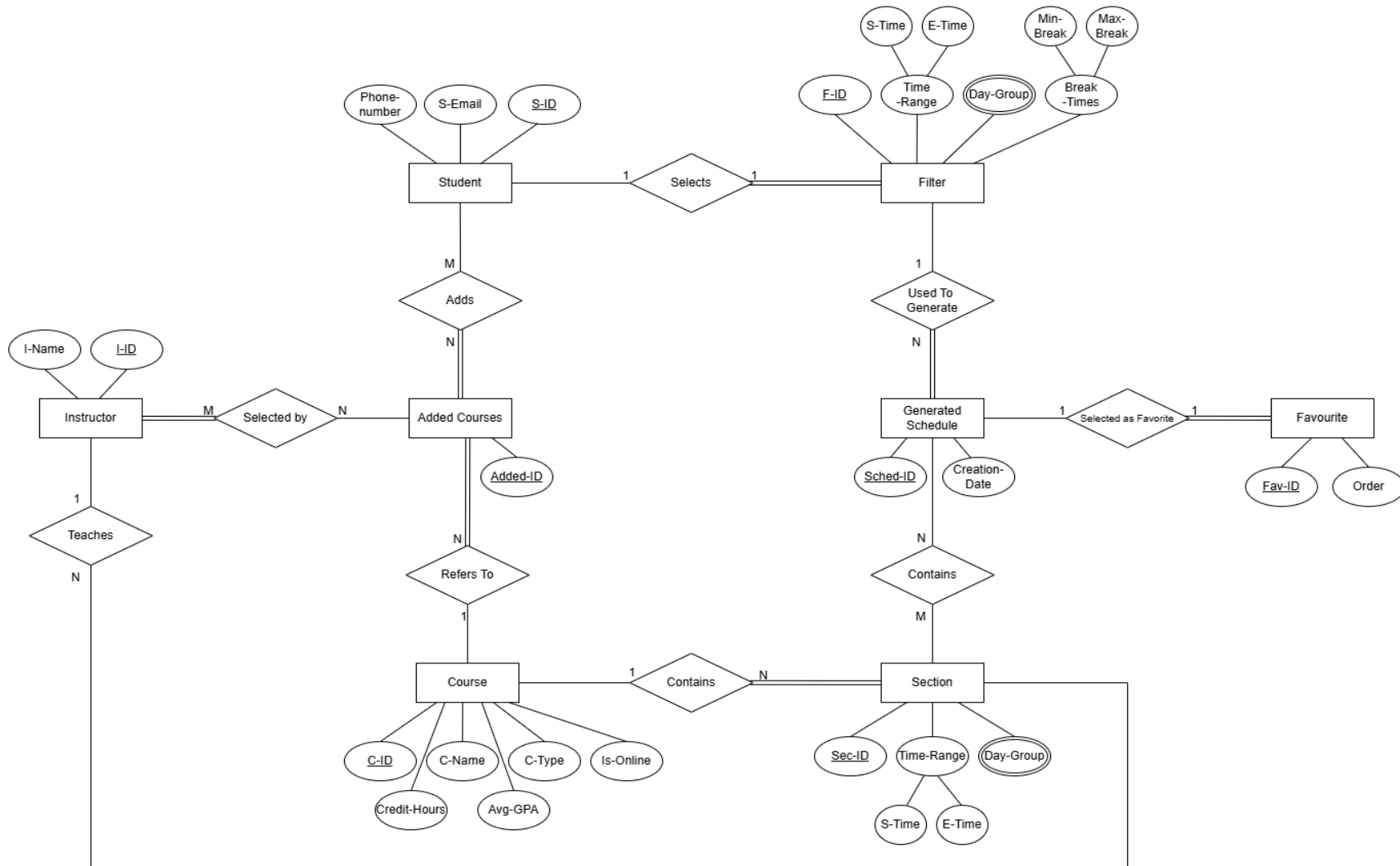


Figure 4.20 ER Diagram showing the entities of the system, their attributes, and their relationships

4.1.4 User Interface Design & Navigation

Table 4.5 show how we navigate through our system, is shows the screen component, its purpose, its key elements and where it navigates to.

Table 4.5 Overview of the User Interface Screens and Navigation Flow

Screen/Page	Purpose	Key Elements	Navigates To
Login Screen	Authenticate user	Username, Password, Login Button	PSUT's Portal
PSUT's Portal Screen	Entering "Smart Scheduler" section	Side Bar Navigation Menu	Remaining Courses Screen
Remaining Courses Screen	Choose Courses	Course Number,Course Title,Course Hours,Requirement Type,Instructor Dropdown Menu, Action	Selected Courses Screen
Selected Courses Screen	Shows Selected Courses	Course Number,Course Title,Course Hours,Requirement Type,Instructor Dropdown Menu, Action	Preferences and Filters Screen
Preferences and Filters Screen	Choose Filters	Preferred Days,Time Range,Break Hours,Credit Hours,Generate Schedules Button	Generated Schedules Screen

Screen/Page	Purpose	Key Elements	Navigates To
Generated Schedules Screen	Shows Generated Schedules Options	Course Number,Course Title,Requirement Type,Instructor, Section,Day,Time,Next Button, Previous Button, Add to Favorite Button, Export to PDF Button	Favorite Schedules Screen
Favorite Schedules Screen	Shows Favorite Schedules	Course Number,Course Title,Requirement Type,Instructor, Section,Day,Time,Next Button, Previous Button, Remove from Favorite Button, Export to PDF Button	

Figure 4.21 shows the system's navigation structure and screen hierarchy, showing how users move between different pages and features.

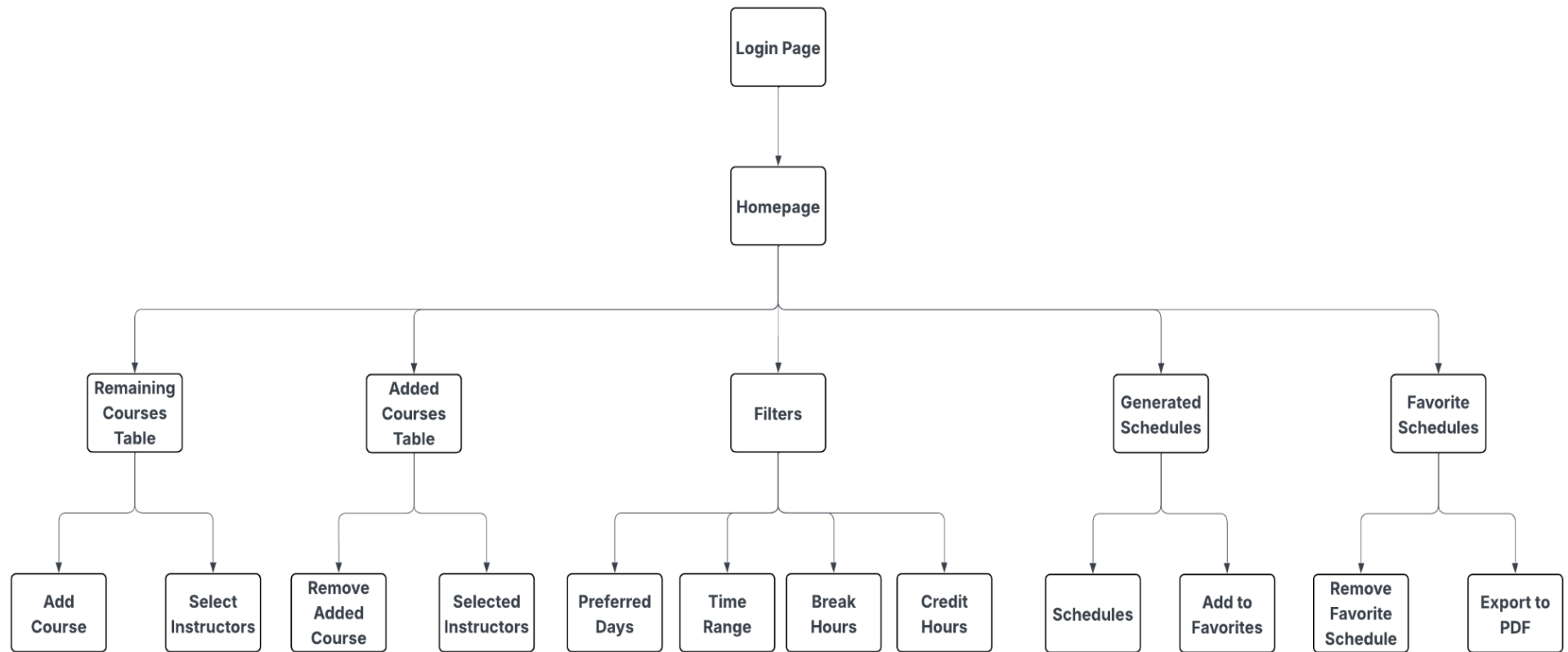


Figure 4.21 Page Tree Diagram showing the student's flow through the page.

Figure 4.22 shows the login page.

The image shows a web browser displaying the login page of Princess Sumaya University for Technology. The page has a light gray header with navigation links: "New Application", "Semester Courses", and "Graduates". A small logo is on the left, and a language selector (UK flag) is on the right. The main content area is a white box with the university's logo and name in English and Arabic. Below the logo is a "Login" section with two input fields: "User Name" and "Password". The "Password" field has a red error message "This field is required." below it. A "Login" button is at the bottom of the form, and a link "Forgot Password? Click Here" is below it. The footer is a dark blue bar with a decorative geometric pattern on the left, links for "ABOUT US", "CONTACT", and "PRIVACY POLICY" in the center, and social media icons (Facebook, Twitter, YouTube, LinkedIn, Instagram, Apple) on the right. A copyright notice is on the far left of the footer.

Princess Sumaya
University
for Technology

جامعة
الأميرة سميرة
للتكنولوجيا

Login

User Name

Password

This field is required.

Login

Forgot Password? [Click Here](#)

ABOUT US CONTACT PRIVACY POLICY

Copyright © 2025 Princess Sumaya University for Technology. All Rights Reserved. Developed by Royal Scientific Society [RSS.jo](#)

f t y in @

Figure 4.22 Login Page

Figure 4.23 shows the remaining courses table and where the student can add the courses.

Student Information

Current Schedule

Exams Schedule

Calc Average

Semesters Courses

Study Plan

Smart Scheduler

Registration

Financial

E-Services

Subscriptions

Survey

Smart Scheduler

Remaining Courses

Course Number	Course Title	Course Hours	Requirement Type	Instructor	Action
11212	Data Structures and Introduction to Algorithms	3	Compulsory Program Requirements	Select Instructors	+ Add Course
20234	Linear Algebra	3	Compulsory School Requirements	Select Instructors	+ Add Course
31374	Life Skills	3	Compulsory University Requirements	Select Instructors	+ Add Course
31311	Scientific Research Methods	3	Elective University Requirements	Select Instructors	+ Add Course
13334	Mobile Application Development	3	Elective Program Requirements	Select Instructors	+ Add Course

Important Note: Make sure to add the courses in **dark blue** (Compulsory Program Requirements) as they're the backbone courses in the major. Not registering them in the considered time might cause graduation delay issues for you.

Figure 4.23 Remaining Courses Table.

Figure 4.24 shows how the student can select his preferred instructors in the remaining courses table.

Student Information

Current Schedule

Exams Schedule

Calc Average

Semesters Courses

Study Plan

Smart Scheduler

Registration

Financial

E-Services

Subscriptions

Survey

Smart Scheduler

Remaining Courses

Course Number	Course Title	Course Hours	Requirement Type	Instructor	Action
11212	Data Structures and Introduction to Algorithms	3	Compulsory Program Requirements	4 selected	+ Add Course
20234	Linear Algebra	3	Compulsory School Requirements		+ Add Course
31374	Life Skills	3	Compulsory University Requirements		+ Add Course
31311	Scientific Research Methods	3	Elective University Requirements	Select Instructors	+ Add Course
13334	Mobile Application Development	3	Elective Program Requirements	Select Instructors	+ Add Course

Important Note: Make sure to add the courses in **dark blue** (Compulsory Program Requirements) as they're the backbone courses in the major. Not registering them in the considered time might cause graduation delay issues for you.

Figure 4.24 Selection of instructors in the remaining courses table.

Figure 4.25 shows the remaining courses table after selecting preferred instructors.

Student Information

Current Schedule

Exams Schedule

Calc Average

Semesters Courses

Study Plan

Smart Scheduler

Registration

Financial

E-Services

Subscriptions

Survey

Smart Scheduler

Remaining Courses

Course Number	Course Title	Course Hours	Requirement Type	Instructor	Action
11212	Data Structures and Introduction to Algorithms	3	Compulsory Program Requirements	4 selected	+ Add Course
20234	Linear Algebra	3	Compulsory School Requirements	2 selected	+ Add Course
31374	Life Skills	3	Compulsory University Requirements	2 selected	+ Add Course
31311	Scientific Research Methods	3	Elective University Requirements	1 selected	+ Add Course
13334	Mobile Application Development	3	Elective Program Requirements	1 selected	+ Add Course

Important Note: Make sure to add the courses in **dark blue** (Compulsory Program Requirements) as they're the backbone courses in the major. Not registering them in the considered time might cause graduation delay issues for you.

Figure 4.25 After selecting the instructors.

Figure 4.26 shows the courses the student added.

Selected Courses

Course Number	Course Title	Course Hours	Requirement Type	Instructors	Action
11212	Data Structures and Introduction to Algorithms	3	Compulsory Program Requirements	Samer Sawalha ▾	 Remove
20234	Linear Algebra	3	Compulsory School Requirements	Samia Bushnaq ▾	 Remove
31374	Life Skills	3	Compulsory University Requirements	Hadeel Alomari ▾	 Remove
31311	Scientific Research Methods	3	Elective University Requirements	Abdul Ghani Albaali ▾	 Remove
13334	Mobile Application Development	3	Elective Program Requirements	Hammam Alakhras ▾	 Remove

Figure 4.26 Added courses table that shows the courses you added.

Figure 4.27 shows the instructors the student selected.

Selected Courses					
Course Number	Course Title	Course Hours	Requirement Type	Instructors	Action
11212	Data Structures and Introduction to Algorithms	3	Compulsory Program Requirements	Samer Sawalha	Remove
20234	Linear Algebra	3	Compulsory School Requirements	Sufian Almajali	Remove
31374	Life Skills	3	Compulsory University Requirements	Ibrahim Albluwi	Remove
31311	Scientific Research Methods	3	Elective University Requirements	Abdul Ghani Albaali	Remove
13334	Mobile Application Development	3	Elective Program Requirements	Hamam Alakhras	Remove

Figure 4.27 Dropdown menu displaying the instructors you selected.

Figure 4.28 shows the filters that the student can choose as his preference.



Preferences & Filters

Preferred Days

☒ Sunday ☒ Monday ☒ Tuesday ☒ Wednesday ☒ Thursday

Time Range

From: To:

Break Hours

Minimum: hours Maximum: hours

Credit Hours

credit hours

[Generate Schedules](#)

Figure 4.28 Filters to input the preferred constraints.

Figure 4.29 Shows how the student can select the time in the time range filter.

The screenshot displays a 'Preferences & Filters' form. On the left is a large dark blue rectangular area. The form contains the following sections:

- Preferred Days:** Checkboxes for Sunday, Monday, Tuesday, Wednesday, and Thursday, all of which are checked.
- Time Range:** Includes 'From' and 'To' input fields. The 'From' field contains '08:00 AM' and the 'To' field contains '03:30 PM'. A time picker overlay is visible, showing a grid of times from 10:28 PM to 04:34 AM. The time '03:30' is highlighted in blue.
- Break Hours:** A 'Minimum' input field with the value '0' followed by the text 'hours'.
- Credit Hours:** An input field with the value '15' followed by the text 'credit hours'.
- Generate Schedules:** A blue button at the bottom of the form.

The time picker overlay shows the following times in a grid:

10	28	PM
11	29	AM
12	30	
01	31	
02	32	
03	33	
04	34	

Figure 4.29 Usable time box for easy choice in the time range field.

Figure 4.30 Shows the numeric box where the student will select his break hours.



Preferences & Filters

Preferred Days

☒ Sunday ☒ Monday ☒ Tuesday ☒ Wednesday ☒ Thursday

Time Range

From: To:

Break Hours

Minimum: hours Maximum:  hours

Credit Hours

credit hours

[Generate Schedules](#)

Figure 4.30 Numeric box for the break hours filed.

Figure 4.31 Shows the numeric box where the student will select his break hours.



Preferences & Filters

Preferred Days

☒ Sunday ☒ Monday ☒ Tuesday ☒ Wednesday ☒ Thursday

Time Range

From: To:

Break Hours

Minimum: hours Maximum: hours

Credit Hours

 credit hours

[Generate Schedules](#)

Figure 4.31 Numeric box for the credit hours field.

Figure 4.32 shows the generated schedules (option 1).

Generated Schedules

< Previous

Schedule Option 1 of 3
Total Hours: 15

Next >

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	1	Ibrahim Albluwi	Sun/Tue/Thu	8:00-9:00 AM
20234	Linear Algebra	Compulsory School Requirements	1	Samia Bushnaq	Sun/Tue/Thu	9:00-10:00 AM
31374	Life Skills	Compulsory University Requirements	1	Hadeel Alomari	Mon/Wed	3:30-5:00 PM
31311	Scientific Research Methods	Elective University Requirements	1	Abdul Ghani Albaali	Sun/Tue/Thu	10:00-11:00 AM
13334	Mobile Application Development	Elective Program Requirements	1	Hamam Alakhras	Mon/Wed	8:00-9:30 AM

☆ Add to Favorites

📄 Export to PDF

Figure 4.32 Schedules generated after clicking generate schedules.

Figure 4.33 shows the generated schedules (option 2).

Generated Schedules

< Previous

Schedule Option 2 of 3
Total Hours: 15

Next >

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	2	Sufian Almajali	Sun/Tue/Thu	9:00-10:00 AM
20234	Linear Algebra	Compulsory School Requirements	2	Fida Mohammed	Mon/Wed	8:00-9:30 AM
31374	Life Skills	Compulsory University Requirements	2	Hadeel Alomari	Mon/Wed	5:00-6:30 PM
31311	Scientific Research Methods	Elective University Requirements	2	Abdul Ghani Albaali	Mon/Wed	9:30-11:00 AM
13334	Mobile Application Development	Elective Program Requirements	2	Hammam Alakhras	Sun/Tue/Thu	8:00-9:00 AM

☆ Add to Favorites

📄 Export to PDF

Figure 4.33 Option 2 of the generated schedules.

Error! Reference source not found. shows the generated schedules (option 3).

Generated Schedules

< Previous

Schedule Option 3 of 3
Total Hours: 15

Next >

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	3	Samer Sawalha	Sun/Tue/Thu	1:00-2:00 PM
20234	Linear Algebra	Compulsory School Requirements	3	Fida Mohammed	Mon/Wed	9:30-11:00 AM
31374	Life Skills	Compulsory University Requirements	3	Arab Husni	Sun/Tue/Thu	3:00-4:00 PM
31311	Scientific Research Methods	Elective University Requirements	3	Abdul Ghani Albaali	Mon/Wed	2:00-3:30 PM
13334	Mobile Application Development	Elective Program Requirements	3	Hamam Alakhras	Sun/Tue/Thu	12:00-1:00 PM

☆ Add to Favorites

📄 Export to PDF

Figure 4.34 Option 3 of the generated schedules.

Figure 4.35 shows how to select a schedule as favorite (option 1).

Generated Schedules

< Previous

Schedule Option 1 of 3
Total Hours: 15

Next >

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	1	Ibrahim Albluwi	Sun/Tue/Thu	8:00-9:00 AM
20234	Linear Algebra	Compulsory School Requirements	1	Samia Bushnaq	Sun/Tue/Thu	9:00-10:00 AM
31374	Life Skills	Compulsory University Requirements	1	Hadeel Alomari	Mon/Wed	3:30-5:00 PM
31311	Scientific Research Methods	Elective University Requirements	1	Abdul Ghani Albaali	Sun/Tue/Thu	10:00-11:00 AM
13334	Mobile Application Development	Elective Program Requirements	1	Hamam Alakhras	Mon/Wed	8:00-9:30 AM

★ Remove from Favorites

📄 Export to PDF

Figure 4.35 Selecting option 1 as favorite.

Figure 4.36 shows how to select a schedule as favorite (option 2).

Generated Schedules

< Previous

Schedule Option 2 of 3
Total Hours: 15

Next >

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	2	Sufian Almajali	Sun/Tue/Thu	9:00-10:00 AM
20234	Linear Algebra	Compulsory School Requirements	2	Fida Mohammed	Mon/Wed	8:00-9:30 AM
31374	Life Skills	Compulsory University Requirements	2	Hadeel Alomari	Mon/Wed	5:00-6:30 PM
31311	Scientific Research Methods	Elective University Requirements	2	Abdul Ghani Albaali	Mon/Wed	9:30-11:00 AM
13334	Mobile Application Development	Elective Program Requirements	2	Hammam Alakhras	Sun/Tue/Thu	8:00-9:00 AM

★ Remove from Favorites

📄 Export to PDF

Figure 4.36 Selecting option 2 as favorite.

Figure 4.37 shows the first option you selected as favorite, along with an option to export it to pdf.

Favorite Schedules

★ Favorite Schedule 1

Total Hours: 15

Export to PDF

Remove

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	1	Ibrahim Albluwi	Sun/Tue/Thu	8:00-9:00 AM
20234	Linear Algebra	Compulsory School Requirements	1	Samia Bushnaq	Sun/Tue/Thu	9:00-10:00 AM
31374	Life Skills	Compulsory University Requirements	1	Hadeel Alomari	Mon/Wed	3:30-5:00 PM
31311	Scientific Research Methods	Elective University Requirements	1	Abdul Ghani Albaali	Sun/Tue/Thu	10:00-11:00 AM
13334	Mobile Application Development	Elective Program Requirements	1	Hammam Alakhras	Mon/Wed	8:00-9:30 AM

Figure 4.37 Displayed favorite schedule 1 with export to pdf feature (for all favorites) and remove favorite feature.

Figure 4.38 shows the second option you selected as favorite, along with an option to export it to pdf.

 **Favorite Schedule 2** Total Hours: 15

Export to PDF

Remove

Course Number	Course Title	Requirement Type	Section	Instructor	Day	Time
11212	Data Structures and Introduction to Algorithms	Compulsory Program Requirements	2	Sufian Almajali	Sun/Tue/Thu	9:00-10:00 AM
20234	Linear Algebra	Compulsory School Requirements	2	Fida Mohammed	Mon/Wed	8:00-9:30 AM
31374	Life Skills	Compulsory University Requirements	2	Hadeel Alomari	Mon/Wed	5:00-6:30 PM
31311	Scientific Research Methods	Elective University Requirements	2	Abdul Ghani Albaali	Mon/Wed	9:30-11:00 AM
13334	Mobile Application Development	Elective Program Requirements	2	Hammam Alakhras	Sun/Tue/Thu	8:00-9:00 AM

Figure 4.38 Displayed favorite schedule 2 with remove favorite feature.

Figure 4.39 shows the message that appears when there is no schedule satisfying the user's requirements.

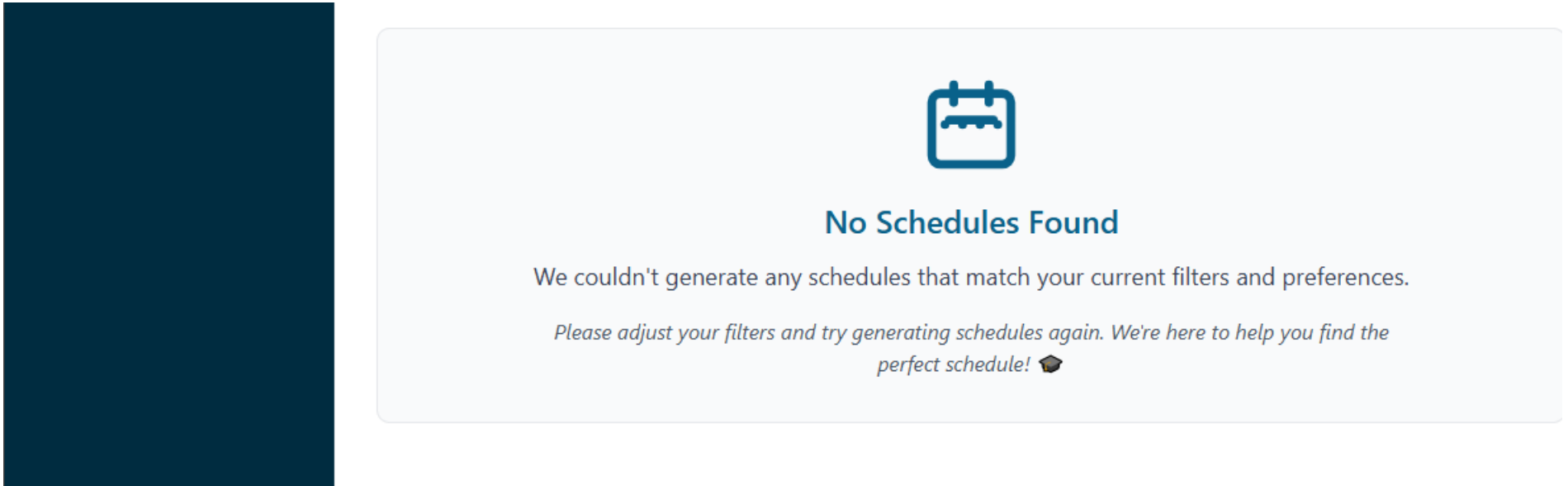


Figure 4.39 No schedule found message when no schedule is generated

Physical Model Design

4.1.5 Reports Design

Table 4.6 shows the report design details for exporting a student's favorite schedule as a PDF file.

Table 4.6 Report Design Specification for Exporting Favorite Schedules as PDF

Report Name	Audience	Inputs/Filters	Layout Fields (Order)	Format
Preferred Schedule	Students	Favorite Schedule	- Course Number - Course Title - Requirement Type - Instructor - Section - Time - Day	PDF

4.1.6 User Interface Design (Physical Specs)

Table 4.7 describes the physical specifications of the website. It shows the primary specs, their accessibility and extra notes describing them.

Table 4.7 Table showing the physical specifications (UI) of the system.

Token/Component	Spec	Accessibility	Notes
Primary Color	#030213	Contrast $\geq 4.5:1$ with foreground	Used for primary buttons, headers, active sidebar items
Secondary Color	#ecef2	Sufficient contrast for backgrounds	Used for secondary actions and subtle UI sections

Token/Component	Spec	Accessibility	Notes
Background	#ffffff	High contrast in light mode, consistent background color across all pages	Global page background
Headings (H1–H4)	Medium weight, responsive sizes	Semantic headings for screen readers	Improves content hierarchy
Font Stack	Inter style, 16px base	Resizable text	Use rem units, consistent typography across components
Button	Primary/Secondary sizes	Focus ring, ARIA labels, keyboard accessible	Disabled and hover states
Form Inputs	Labels, hints, error text	Field landmarks, label association	Client/server validation
Tables (Courses / Schedules)	Card-style rows, muted separators	Readable contrast, structured headers	Used in Remaining, Selected, Generated, Favorite schedules
Dropdowns (Instructor Select)	Popover-based select menus	ARIA roles, keyboard navigation	Used to select instructors in remaining courses and to display what you selected in the added courses.
Border Radius	Radius around 0.625 rem (10px)	Consistent shape language	Used across cards, buttons, inputs

4.1.7 Database Design (3NF)

Figure 4.40 shows the structure of the database in our system. It is in the third normal form. It also shows the relationships between the tables in the database.

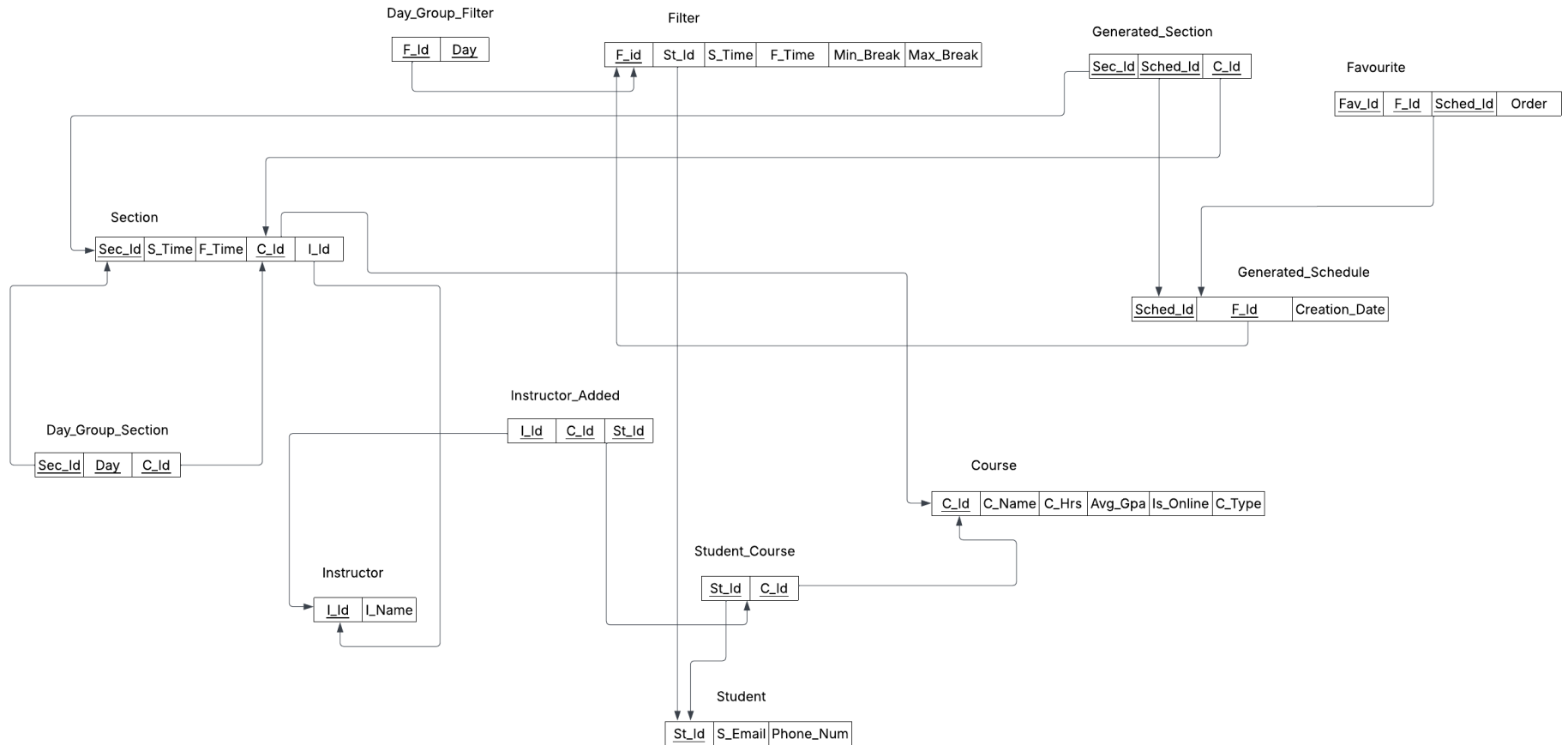


Figure 4.40 Schema showing the actual physical structure of the system's database in the third normal form

Table 4.8 shows the normalization details of each table in the system's database, along with a proof to emphasize that each table is in the normal form.

Table 4.8 Physical schema normalization details.

Table	1NF	2NF	3NF	Notes/Proof
Student	Atomic Columns	No partial dependencies	No transitive deps	Email is unique. All attributes depend only on St_id
Course	Atomic Columns	No partial dependencies	No transitive deps	All attributes depend only on C_id
Instructor	Atomic Columns	No partial dependencies	No transitive deps	Instructor name depends only on I_id
Section	Atomic Columns	Full composite key dependency	No transitive deps	All attributes depend on both Sec_id and C_id
Student_course	Atomic Columns	Full composite key dependency	No transitive deps	All attributes are primary keys
Instructor_added	Atomic Columns	Full composite key dependency	No transitive deps	All attributes are primary keys
Filter	Atomic Columns	No partial dependencies	No transitive deps	Filter criteria depend only on F_id

Table	1NF	2NF	3NF	Notes/Proof
Day_group_filter	Atomic Columns	Full composite key dependency	No transitive deps	All attributes are primary keys
Generated_schedule	Atomic Columns	Full composite key dependency	No transitive deps	Creation date depends on Sched_id, F_id
Generated_section	Atomic Columns	Full composite key dependency	No transitive deps	All attributes are primary keys
Day_Group_Section	Atomic Columns	Full composite key dependency	No transitive deps	All the attributes are primary keys.
Favourite	Atomic Columns	Full composite key dependency	No transitive deps	Ranking (order) depends on Fav_id, F_id, Sched_id

Chapter 5 Implementation

5.1 Introduction

This chapter outlines the details of the Smart Scheduling System. It describes the technologies, frameworks, programming languages, and tools used in development and explains why they were chosen. The chapter also covers the coding standards, algorithms, system architecture, database implementation, integration process, and security measures. Additionally, it highlights the features that were implemented, those that were postponed, and the methods used to improve system performance and usability.

5.2 Development Tools, Languages, and APIs Statement

Table 5.1 showing Development Tools, Languages, and APIs used in the project.

Tool/Language/API	Purpose/Usage	Reason for Selection
C# / ASP.NET Core	Backend development and business logic	Provides strong performance, scalability, and seamless integration with databases
Entity Framework Core	ORM and database communication	Simplifies database operations and reduces SQL complexity
Oracle Database	Relational database management	Reliable for handling structured academic data and relationships
HTML5	Structure of web pages	Standard language for frontend development
CSS3	Styling and responsive design	Enhances user interface appearance and responsiveness
JavaScript	Frontend interactivity and dynamic behavior	Enables real-time interaction and dynamic content updates
JSON	Used for communication between frontend and backend APIs	Lightweight, widely supported, and efficient for exchanging structured data between the frontend and backend.

Tool/Language/API	Purpose/Usage	Reason for Selection
Gemini API	AI-based schedule ranking and recommendation	Provides intelligent schedule prioritization
Git & GitHub	Version control and collaboration	Tracks changes and supports teamwork
Visual Studio 2022	Main development environment	Powerful IDE with debugging and ASP.NET support
SQL Plus	Database management and query execution	Simplifies Oracle database interaction
jsPDF & html2canvas	PDF export generation	Enables exporting schedules with styled formatting
Postman	API testing	Verifies backend API functionality

5.3 System Implementation Overview

The Smart Scheduling System was implemented using a modular structure that separates frontend, backend, and database layers.

The implemented system contains:

- main backend modules
- More than 35 classes and models
- Approximately 6,000 lines of source code
- RESTful API endpoints for schedule generation and student operations
- Multiple frontend pages for course selection, schedule generation, and favorite schedules
- Oracle relational database containing interconnected academic tables

The database includes tables such as:

- STUDENT
- COURSE
- SECTION
- STUDENT_COURSE
- DAY_GROUP_SECTION
- FAVORITE_SCHEDULE

The frontend contains dynamic interfaces that allow students to:

- Select courses
- Generate schedules
- Save favorite schedules
- Export schedules to PDF

View online and blended course information

5.4 Coding Conventions

To ensure readability, maintainability, and teamwork consistency, several coding conventions were followed during implementation.

1. Naming Conventions:

- Class names use PascalCase
 - Example: SchedulerController
- Variable and method names use camelCase
 - Example: generateSchedules()
- Database columns use uppercase naming consistent with Oracle conventions.

2. Code Organization:

- Backend logic separated into:
 - Controllers
 - Services
 - DTOs
 - Models
 - Database Context
- Frontend scripts separated into reusable JavaScript functions.

3. Documentation and Comments :

Important methods and complex logic sections include comments explaining their purpose.

Example: // Generates all possible valid schedules.

4. Version Control:

- Git and GitHub were used for: Team collaboration, tracking modifications, managing branches, preventing code conflicts.
- Sensitive configuration files such as appsettings.json were excluded using .gitignore.

5.5 Core Algorithms

5.5.1 Schedule Generation Algorithm

Problem Solved: Students often struggle to manually create schedules that satisfy:

- Credit hour constraints
- Time conflict prevention
- Course availability
- Prerequisite completion

Main Idea: The algorithm generates all valid combinations of selected course sections while ensuring:

- No time overlap
- Credit hour limits are respected
- Prerequisites are satisfied

Why It Was Chosen: This approach ensures flexibility and provides students with multiple valid scheduling options instead of a single fixed schedule.

Simplified Pseudocode:

```
Input selected courses
Generate all possible section combinations
For each combination:
    Check time conflicts
    Check credit hour constraints
    Check prerequisites
If valid:
    Add to valid schedules list
Return all valid schedules
```

5.5.2 AI Schedule Ranking Algorithm

Problem Solved: Students may receive many valid schedules, making it difficult to choose the best one.

Main Idea: The Gemini API evaluates generated schedules based on:

- Student GPA
- Course difficulty
- Previous grades
- Schedule balance
- Online/blended preferences

Why It Was Chosen: AI ranking provides personalized recommendations instead of random ordering.

Simplified Workflow:

Generate valid schedules

Send schedules to Gemini API

Receive ranked schedules

Display ranked schedules to student (Best to worst)

5.6 Implemented and Deferred Features

Feature ID	Feature Description	Priority	Implementation Status
FR-1	User Login & Authentication	Essential	Implemented
FR-2	Retrieve Remaining Courses	Essential	Implemented
FR-3	Add Course to “Added Courses” List	Optional	Implemented
FR-4	Select Instructor Preference	Optional	Implemented
FR-5	Set Time Range	Optional	Implemented
FR-6	Set Break Time	Optional	Implemented
FR-7	Select Preferred Attendance Days	Optional	Implemented
FR-8	Set Credit Hours	Essential	Implemented
FR-9	Generate Non-Conflicting Schedules	Essential	Implemented
FR-10	Rank Generated Schedules	Optional	Implemented
FR-11	Mark Schedule as Favorite	Optional	Implemented
FR-12	Export Schedule (PDF)	Optional	Implemented

5.7 System Architecture and Deployment Details

The Smart Scheduling System follows a three-tier architecture:

1. Presentation Layer (Frontend)
 2. Business Logic Layer (Backend)
 3. Data Layer (Database)
- Frontend: was developed using:
 - HTML
 - CSS
 - JavaScript

It provides a responsive interface for student interaction.

- Backend: was developed using ASP.NET Core Web API and handles:
 - Schedule generation
 - Authentication
 - Database communication
 - AI integration
- Database Layer: Oracle Database stores all academic and scheduling information.
- Deployment: The system was tested locally using:
 - IIS Express
 - ASP.NET local server

5.8 Database Design and Implementation

The database uses a relational model to ensure consistency and scalability.

- Main Relationships:
 - One student can register for multiple courses
 - One course can contain multiple sections
 - Sections can belong to multiple day groups
- Database Features
 - Primary Keys: Used to uniquely identify records.
 - Foreign Keys: Used to maintain referential integrity.
 - Normalization: The database was normalized to Third Normal Form (3NF) to reduce redundancy.

Example SQL Query:

```
SELECT          c.C_NAME,          s.SEC_ID
FROM          COURSE          c
JOIN SECTION s ON c.C_ID = s.C_ID;
```

- Constraints for data validation.
- Indexed columns for faster searching.
- Oracle Data Pump support for backup/export.

5.9 Integration Between Components

The system components communicate using RESTful APIs.

Workflow Example:

1. Student selects courses from frontend
2. Frontend sends HTTP request to backend
3. Backend validates data
4. Backend communicates with Oracle database
5. Schedule generation algorithm executes
6. AI ranking is performed
7. Results returned as JSON response
8. Frontend displays generated schedules

JSON format was used for data exchange because it is lightweight and easy to process.

5.10 Error Handling and Security

Error Handling: The system uses structured exception handling with try-catch blocks.

Example:

```
try
{
    var totalWatch = Stopwatch.StartNew();

    var generateWatch = Stopwatch.StartNew();

    var schedules = await _svc.GenerateSchedulesAsync(req.FilterId, studentId);

    generateWatch.Stop();

    Console.WriteLine($"TIME {generateWatch.ElapsedMilliseconds} ms" - GenerateSchedulesAsync:
    {generateWatch.ElapsedMilliseconds} ms");

    Console.WriteLine($"COUNT - Generated schedules: {schedules.Count}");

    if (schedules.Count > 0)
    {
        var dbWatch = Stopwatch.StartNew();

        var student = await _context.Students.FirstOrDefaultAsync(s => s.StId == studentId);

        if (student == null)
        {
            return BadRequest("Student not found.");
        }

        var studentGrades = await _context.StudentGrades
            .Where(g => g.StId == studentId)
            .Select(g => new
            {
                g.CourseName,
```

```

        g.CourseGpa,
        g.PassFlag
    })
    .ToListAsync();
    dbWatch.Stop();
    Console.WriteLine($"TIME - Load student + grades: {dbWatch.ElapsedMilliseconds} ms");
    Console.WriteLine($"COUNT - Student grades: {studentGrades.Count}");
    var requestData = new
    {
        StudentId = studentId,
        GPA = student.Gpa, // from STUDENT table
        CompletedCourses = studentGrades,
        Schedules = schedules
    };
    var geminiWatch = Stopwatch.StartNew();
    var sortedIds = await _geminiRanker.RankSchedulesAsync(requestData);
    geminiWatch.Stop();    Console.WriteLine($"TIME - Gemini ranking:
{geminiWatch.ElapsedMilliseconds} ms");
    var sortedSchedules = sortedIds
        .Select(id => schedules.FirstOrDefault(s => s.SchedId == id))
        .OfType<ScheduleResultDto>()
        .ToList();
    var remainingSchedules = schedules
        .Where(s => !sortedIds.Contains(s.SchedId))
        .ToList();

```

```

        schedules = sortedSchedules!
            .Concat(remainingSchedules)
            .ToList();
    }

    var msg = schedules.Count == 0
        ? "No conflict-free schedules found. Try relaxing your filters."
        : $"{schedules.Count} schedule(s) generated.";

    totalWatch.Stop();

    Console.WriteLine($"TIME - TOTAL API: {totalWatch.ElapsedMilliseconds} ms");

    return Ok(ApiResponse<List<ScheduleResultDto>>.Ok(schedules, msg));
}

catch (Exception ex)
{
    var msgs = new List<string>();

    var cur = ex;

    while (cur != null)
    {
        msgs.Add(cur.GetType().Name + ": " + cur.Message);

        cur = cur.InnerException;
    }

    var full = string.Join(" --> ", msgs);

    _logger.LogError("Generate failed: {E}", full);

    return StatusCode(500, ApiResponse<List<ScheduleResultDto>>.Fail(full));
}

```

Security Mechanisms:

- Authentication: Secure login validation was implemented.
- SQL Injection Prevention: Entity Framework parameterized queries prevent malicious SQL input.
- Logging: Errors and debugging information are logged for maintenance and monitoring purposes.

5.11 Performance Optimization

Several optimization techniques were applied.

- Query Optimization
 - Indexed frequently used columns
 - Optimized joins and filtering
- Efficient Schedule Filtering
 - Invalid schedules are discarded early to reduce processing overhead.
- Database Normalization
 - Reduced redundancy and improved consistency.
- Frontend Optimization
 - Dynamic rendering
 - Reduced unnecessary refreshes
 - Efficient JavaScript filtering

5.12 Challenges and Solutions

Table 5.3 showing the challenges faced during the implementation and the solutions used.

Challenge	Solution
Large number of schedule combinations	Implemented filtering and validation during generation
Time conflict handling	Developed conflict detection algorithm
API response ordering	Added AI-based sorting and debugging
PDF export formatting	Switched to HTML + CSS rendering with jsPDF
Online course mapping issues	Fixed DTO and JSON serialization mapping
Team collaboration conflicts	Used Git branching and .gitignore

5.13 Ethical and Sustainability Considerations

- Ethical considerations: The system respects ethical software development principles by protecting student data and ensuring privacy.

Measures include:

- Secure handling of academic records
 - Restricted access to sensitive information
 - Prevention of unauthorized database access
- Sustainability considerations: The system contributes to sustainability by reducing paper-based scheduling and improving digital academic management.

5.14 User Interface Implementation

The user interface of the Smart Scheduling System was created as a web-based interface using HTML, CSS, and JavaScript. The frontend was designed to help students easily interact with the system. They can select their scheduling preferences, generate schedules, save favorite schedules, and export chosen schedules as PDF files.

- Main Interface Components:

Table 5.4 showing each component in the interface and how it is used.

Interface Component	Description
Login Page	Allows the student to enter university id for login.
Remaining Courses Page	Displays the courses that the student still needs to complete.
Added Courses List	Displays the courses added before generating schedules.
Instructor Preference Dropdown	Allows the student to choose preferred instructors for selected courses.

Time Range Filter	Allows the student to select the start and finish time.
Break Time Filter	Allows the student to choose the minimum and maximum break duration.
Attendance Days Selection	Allows the student to choose preferred attendance days.
Credit Hours Selection	Allows the student to set the desired number of credit hours.
Generated Schedules Page	Displays non-conflicting schedules and ranked schedule options.
Favorite Schedules Page	Allows students to save and access preferred schedules.
PDF Export Button	Allows students to export a selected schedule as a downloadable PDF file.

▪ Technologies Used in the Interface:

Table 5.5 showing technologies used in the interface and how it is used.

Technology	Usage in the User Interface
HTML	Used to structure pages, forms, buttons, tables, and schedule cards.
CSS	Used to style the interface, improve readability, and organize layout spacing.
JavaScript	Used for dynamic actions such as adding courses, applying filters, displaying generated schedules, saving favorites, and triggering PDF export.
jsPDF	Used to generate downloadable PDF versions of selected schedules.

html2canvas	Used to capture the visual schedule layout before converting it into a PDF.
JSON	Used to exchange schedule and course data between the frontend and backend APIs.

5.15 Summary

This chapter has looked at how the Smart Scheduling System was built. It included the technologies used, the design of the system, algorithms, database design, security measures and ways to improve its performance. The implemented features demonstrate how the proposed design became a useful scheduling tool to help students create effective academic schedules. In addition, the completion of the implementation phase enabled extensive testing and evaluation to assess the system's functionality, performance, reliability, and usability.

Chapter 6 Testing

6.1 Testing Approach

Several testing approaches were used to ensure the correctness, reliability, and usability of the Smart Scheduling System.

- **Unit Testing:** Individual backend functions and validation methods were tested separately. Examples include:
 - Credit hour validation
 - Time conflict checking
 - Prerequisite checking
- **Integration Testing:** It verified communication between:
 - Frontend and backend
 - Backend and database
 - Backend and Gemini API
- **System Testing:** The complete system was tested as a whole to ensure all components functioned correctly together.
- **User Acceptance Testing (UAT):** Students tested the system by generating schedules and providing feedback on usability and the quality of the schedules.
- **Performance Testing:** The response time was tested with large course selections to generate the schedule to verify the acceptable response time.
- **Security Testing:** Authentication and input validation was tested to prevent unauthorized access and invalid data submission.

6.2 Testing Tools

6.2.1 General and Functional Testing

Table 6.1 showing general and functional testing tools used and why it is used.

Tool	Purpose
Postman	API testing
Browser Developer Tools	Frontend debugging
Visual Studio Debugger	Backend debugging
Console Logging	Runtime debugging

6.2.2 Security Testing

Table 6.2 showing security testing tools used and why it is used.

Tool	Purpose
OWASP Principles	Security validation
Entity Framework Validation	SQL injection prevention

6.2.3 Code Quality and Static Analysis

Table 6.3 showing tools used for Code Quality and Static Analysis and why it is used.

Tool	Purpose
Visual Studio Analyzer	Detect code issues
GitHub	Version tracking and code review

6.3 Tested Features

Table 6.4 showing how each component in the system is tested.

Feature	Testing Type	Tool Used	Status	Notes
User Login	Unit & Integration Testing	Postman, Visual Studio Debugger	✓ Tested	Verified authentication and access validation
Retrieve Remaining Courses	Integration Testing	Postman	✓ Tested	Correct remaining courses retrieved from database
Add Courses to List	Functional Testing	Browser Developer Tools	✓ Tested	Added courses displayed correctly
Instructor Preference Filtering	Functional Testing	Browser Developer Tools	✓ Tested	Sections filtered according to selected instructor
Time Range Filtering	Functional Testing	Browser Developer Tools	✓ Tested	Only classes within selected time range displayed
Break Time Filtering	Functional Testing	Browser Developer Tools	✓ Tested	Break constraints applied correctly
Attendance Days Filtering	Functional Testing	Browser Developer Tools	✓ Tested	Schedules matched selected attendance days
Credit Hours Validation	Unit Testing	Visual Studio Debugger	✓ Tested	Credit-hour constraints validated correctly

Schedule Generation	System Testing	Visual Studio Debugger	✓ Tested	Non-conflicting schedules generated successfully
AI Schedule Ranking	Integration Testing	Postman, Gemini API	✓ Tested	Ranked schedules returned correctly
Favorite Schedules	Functional Testing	Browser Developer Tools	✓ Tested	Favorite schedules saved and retrieved successfully
PDF Export	System Testing	Browser Testing	✓ Tested	Schedules exported successfully as PDF files

Screenshots for testing:

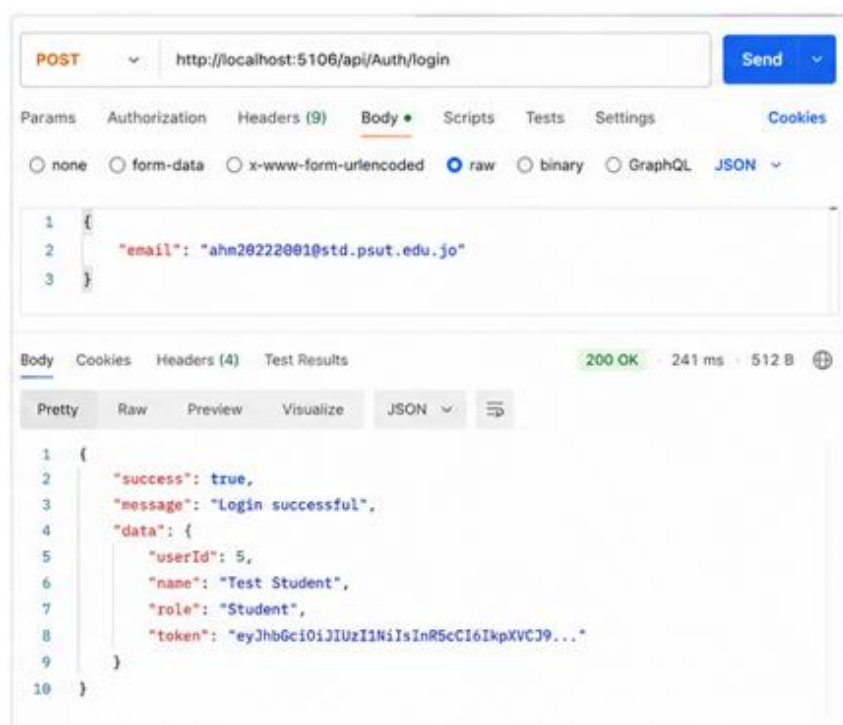


Figure 6.1 showing User Login API Testing Using Postman.

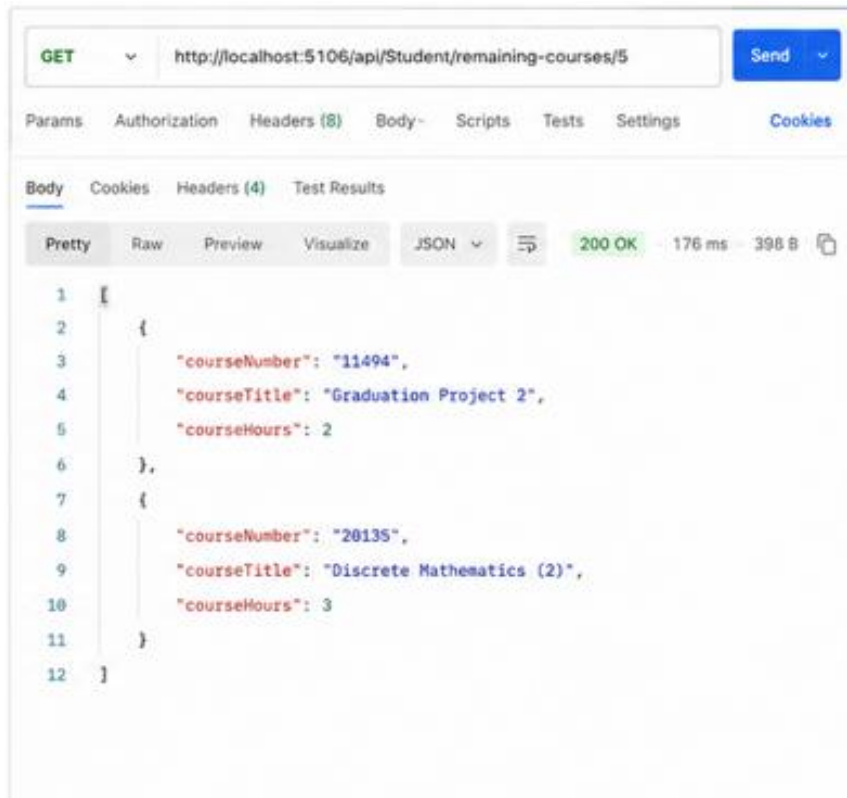


Figure 6.2 showing Remaining Courses Retrieval API Test.

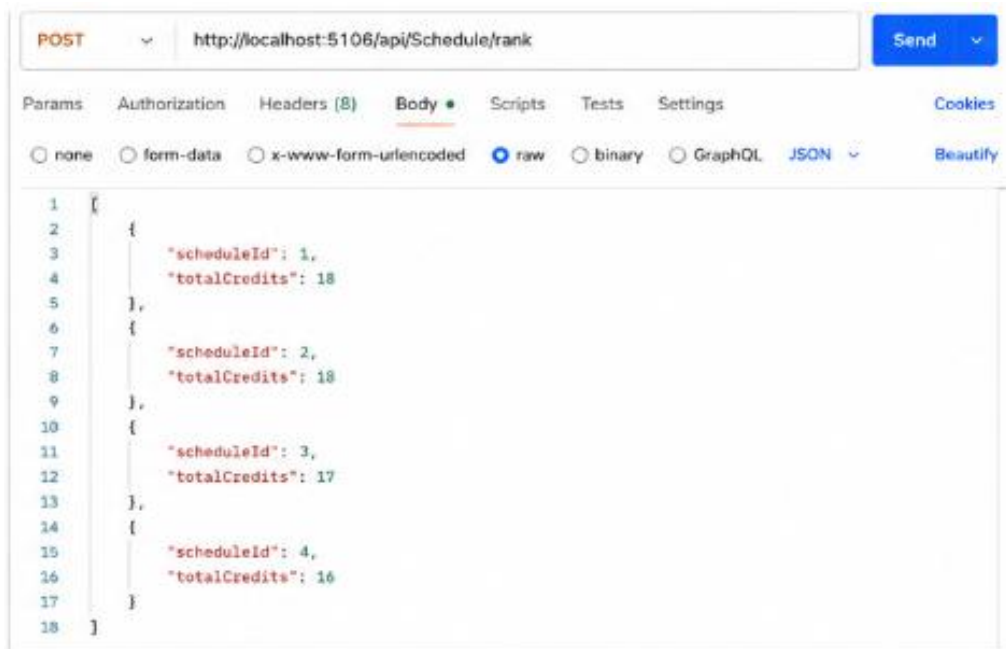


Figure 6.3 showing AI Schedule Ranking API Testing (Request).

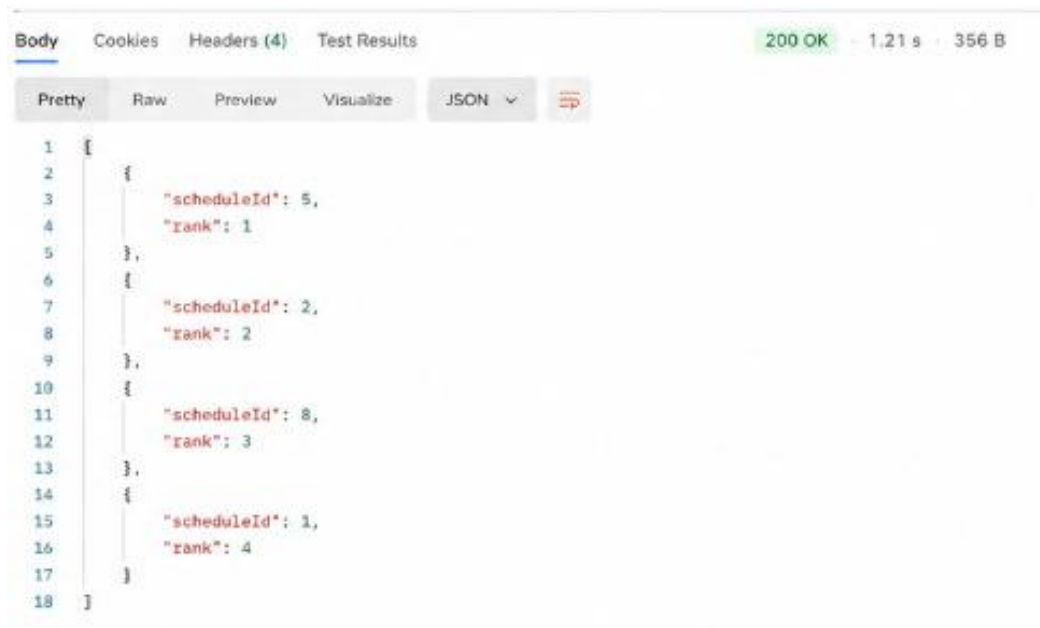


Figure 6.4 showing AI Schedule Ranking API Testing (Response).

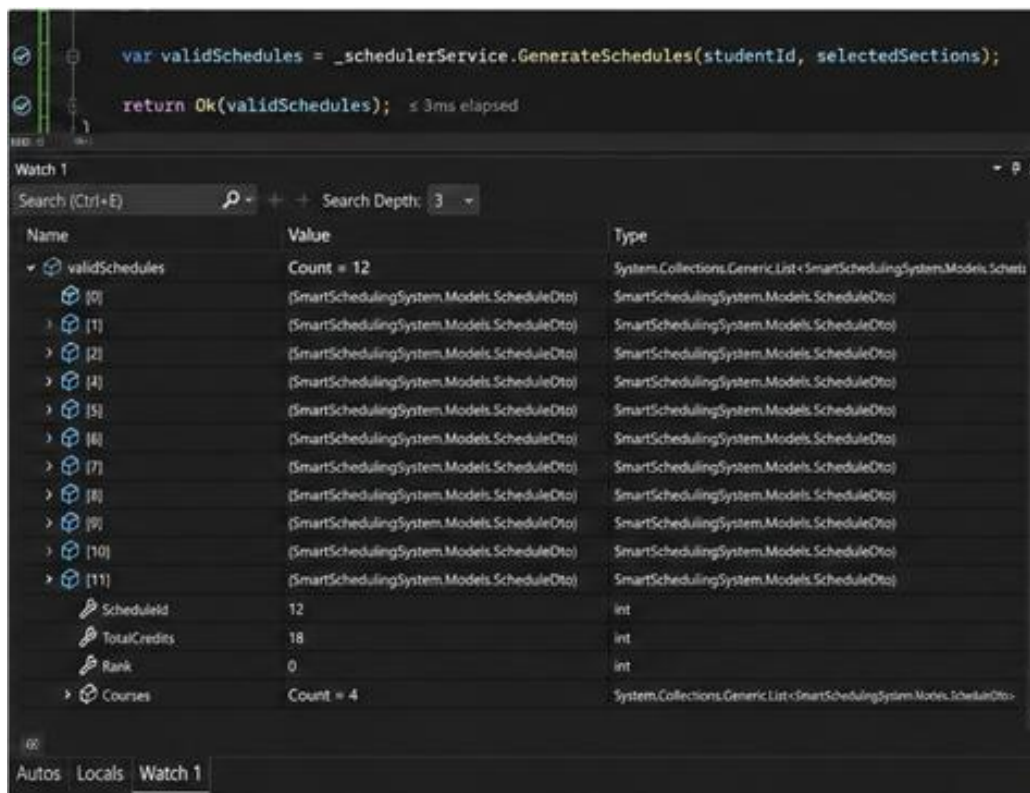


Figure 6.5 showing Schedule Generation Debugging Using Visual Studio.

Test Name	Duration
SmartSchedulingSystem.Tests (12)	14 ms
SmartSchedulingSystem.Tests.Services (12)	5 ms
SchedulerServiceTests (12)	3 ms
CheckCreditHours_ValidHours_ShouldReturnTrue	3 ms
CheckCreditHours_ExceedingHours_ShouldReturnFalse	2 ms
CheckPrerequisites_Met_ShouldReturnTrue	2 ms
CheckPrerequisites_NotMet_ShouldReturnFalse	8 ms
CheckTimeConflict_ShouldReturnFalse_WhenNoConflict	7 ms
CheckTimeConflict_ShouldReturnTrue_WhenConflictExists	4 ms
GetRemainingCourses_ShouldReturnOnlyIncompleteCourses	6 ms
GetSectionsByCourse_ShouldReturnCorrectSections	5 ms
MapSchedule_ShouldMapScheduleCorrectly	3 ms
SeedDatabase_ShouldInsertData	3 ms
ValidateSchedule_ShouldReturnFalse_WhenTimeConflict	6 ms
ValidateSchedule_ShouldReturnTrue_WhenValid	4 ms

Figure 6.6 Unit Testing Results in Visual Studio Test Explorer.

Test run finished: 12 Tests (12 Passed, 0 Failed, 0 Skipped) run in 153 ms

Test	Duration	Traits	Error Message
SmartSchedulingSystem.Tests (12)	153 ms		
SmartSchedulingSystem.Tests.Services (12)	153 ms		
SchedulerServiceTests (12)	153 ms		
CheckCreditHours_ValidHours_ShouldReturnTrue	14 ms		
CheckCreditHours_ExceedingHours_ShouldReturnFalse	5 ms		
CheckPrerequisites_Met_ShouldReturnTrue	8 ms		
CheckPrerequisites_NotMet_ShouldReturnFalse	3 ms		
CheckTimeConflict_ShouldReturnFalse_WhenNoConflict	3 ms		
CheckTimeConflict_ShouldReturnTrue_WhenConflictExists	8 ms		
GetRemainingCourses_ShouldReturnOnlyIncompleteCourses	4 ms		
GetSectionsByCourse_ShouldReturnCorrectSections	6 ms		
MapSchedule_ShouldMapScheduleCorrectly	5 ms		
SeedDatabase_ShouldInsertData	3 ms		
ValidateSchedule_ShouldReturnFalse_WhenTimeConflict	6 ms		
ValidateSchedule_ShouldReturnTrue_WhenValid	4 ms		

Figure 6.7 Successful Execution of Backend Unit Tests Using Visual Studio Test Explorer.

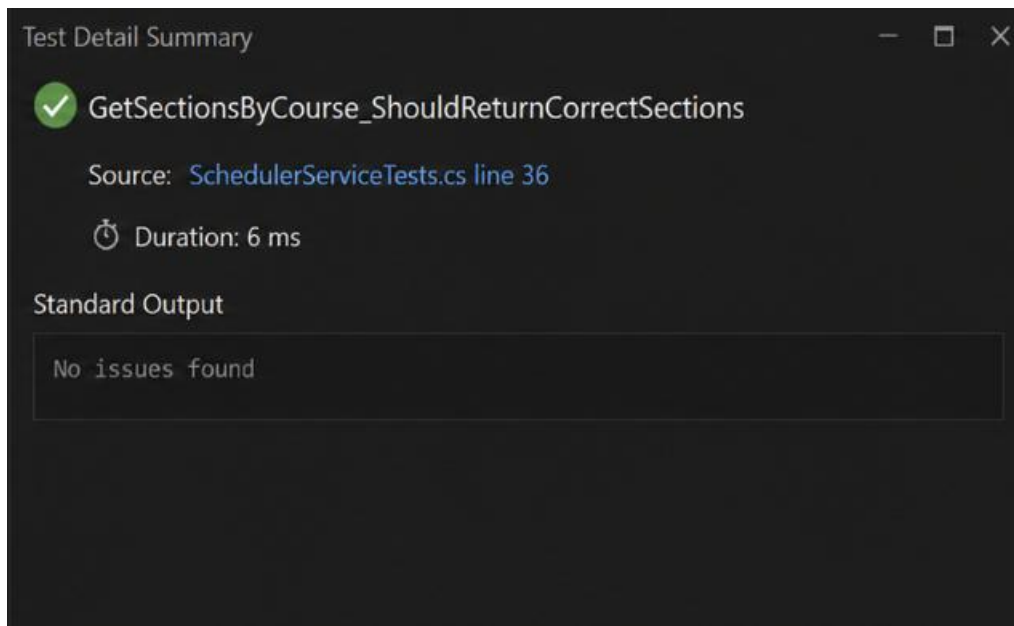


Figure 6.8 Unit Test Result Displayed in Visual Studio Test Explorer.

Bugs found during implementation and solutions:

Table 6.5 showing the main bugs found during implementation and how it was solved.

Bug	Cause	Solution
Some generated schedules contained overlapping lecture times	The conflict-checking logic did not correctly compare all lecture times	The schedule generation algorithm was updated to improve time-conflict validation
Favorite schedules disappeared after page refresh	Favorite schedules were temporarily stored only on the frontend	Persistent database storage was added for favorite schedules
PDF export occasionally generated incomplete schedule files	The schedule was exported before it fully loaded on the page	The export process was modified to wait until the schedule finished loading before generating the PDF

6.4 Discussion

- During testing process, we found several issues and worked hard to resolve it.

Two major bugs were found:

1. PDF Export Failure: When refreshing the page, and clicking the export to pdf button, it says: “Schedule not found”. The reason was that the export function only checked generated schedules, ignoring persisted favorites. This was solved by changing the export function to search both collections
2. Online Course Mapping Issue: All courses were shown as “No”, and the cause was a frontend mapping issue with JSON serialization. This was fixed by modifying the JavaScript mapping logic to allow for camelCase serialization.

- Testing Impact: Testing significantly improved:
 - System stability
 - Schedule accuracy
 - User experience
 - Error handling

Moreover, testing helped in optimizing database queries and improved frontend responsiveness.

Chapter 7 Conclusion

7.1 Summary of the Work

The target of this project is to design and implement a Smart Student Scheduling System that simplifies the process of creating course schedules manually and reduce the stress students face during the registration period. Many students struggle while building their schedules, since they try to avoid time conflicts and they try to generate schedules considering their personal preferences such as preferred days, class times, and instructors. This process is often time-consuming, confusing, and prone to errors. To address these problems, we introduced a web-based system that automates schedule creation while considering the student's individual preferences.

The system allows the students to input their required courses and customize their schedule using filters such as preferred days, time ranges, break durations, and instructor choices. Then it generates multiple clash-free schedules and ranks them based on how well they match the student selected preferences. Moreover, with the integration of our system to the PSUT portal and providing a simple user-friendly interface, this will enable students to easily view, compare and choose their schedules.

7.2 Achievements and Improvements

The system achieved its main goals:

- Made the schedule generation process automatic.
- Generated schedules without time conflicts.
- Allowed the students to generate schedules based on their preferences.
- Produced multiple schedule options.
- Reduced the student's effort and time.
- Helped the students make fast and better decisions.

Compared with the first plan, the team added multiple improvements while building the system:

- We made the interface simpler and easier to use.
- We improved the algorithm so it gives results quickly.
- We tested the system many times to ensure everything works properly.
- We made the system parts work better together.

7.3 Challenges and Lessons Learned

The During the project, we faced some challenges such as designing the database, connecting the system parts together, and building the scheduling algorithm. Because of that, we tested the system many times, fixed errors, and tried different solutions until the system worked correctly.

We learned a lot from this project. We became better in coding and also in working together. Also, we noticed that if we don't organize our work or talk clearly with each other, things get messy fast.

7.4 Future Work

Potential areas for future enhancement include:

- Add notifications for schedule changes
- Compare schedules side by side
- Add a help button that connects students with their academic advisor.
- Let students get feedback on their selected schedule
- Allow editing schedules instead of making a new one

7.5 Final Remarks

In Conclusion, this project shows that working step by step and following a clear plan can lead to a useful system. The Smart Student Scheduling System helped in making schedule planning easier. It can still be improved more in the future with new features.

Appendix A

Documentation Changes

A.1 Overview

This appendix summarizes the major differences between the initial project documentation and the final implemented system. It shows the changes and improvements made during the development process.

A.2 Summary of Key Changes

Aspect	Expected (Planned in Documentation)	Actual (Implemented in Final Version)	Reason / Justification for Change
System Architecture	Basic frontend and backend structure	Expanded architecture with additional components and better organization	To improve system performance and maintainability
User Interface	Simple design with limited interactivity	Responsive and interactive interface	To provide a better user experience
Database Structure	Basic database structure for courses and students only	Expanded database structure with additional tables for filters, schedules, favorites, and other features	To support more system features and improve data organization
Testing Approach	Basic testing	Integration and performance testing	To ensure system reliability and stable performance
Documentation	Basic explanations and brief overview	Added diagrams, screenshots, and detailed explanations	To improve clarity and understanding

A.3 Discussion

During the development process of our project, some changes were made based on our supervisor feedback and system needs. These changes improved our Smart Student Schedule System and made it easier to use.

At first, the database structure was expanded to include more tables such as filters, favorites, generated schedules, etc. Moreover, the user interface improved to be more interactive and easier to use, so the student can have a better experience. Also, the system architecture enhanced to improve the system performance. In addition, the documentation was updated to include details, diagrams and screenshots that reflect the final version of the system.

A.4 Conclusion

This appendix explains the changes made between the planned system and the final implemented version. Each modification was made to improve the performance and to enhance the student experience, while still meeting the original project objectives.

Overall, all changes and enhancements helped the system to be more organized, efficient, and easier for students to use.

References

- [1] Coursicle, “Plan your schedule and get into classes,” *Coursicle*, n.d. [Online]. Available: <https://www.coursicle.com/>. Accessed: Oct. 29, 2025.
- [2] M. Saeed and A. Fahim, “Designing a smart system for building student’s schedule-An application at Neelain University,” *International Journal of Advanced and Applied Sciences*, pp. 82–94, 2021.
- [3] YouScheduler, “College course scheduler & analytics,” *YouScheduler*, n.d. [Online]. Available: <https://www.youscheduler.com/>. Accessed: Oct. 29, 2025.
- [4] UniScheduler, “Uni scheduling made simple,” *UniScheduler*, n.d. [Online]. Available: <https://www.unischeduler.com/>. Accessed: Oct. 29, 2025.